

MetaSignal

Ground Truth / Interview Defense Master Pack v2.0

Complete Edition — Sections 0–17

Sidharth Kriplani

All claims source-verified from repo files · Not for redistribution

Label	Meaning
[IMPLEMENTED]	Code exists, runs, produces verified artifact
[SIMULATED]	Deliberately synthetic; simulation logic is real code
[PARTIALLY VERIFIED]	Code exists; verification relies on internal consistency
[FUTURE]	Not built — explicitly absent from repo

MetaSignal Ground Truth / Interview Defense Master Pack v2.0

Complete Edition — Sections 0–17

Project: MetaSignal Platform

Author: Sidharth Kriplani

Document version: v2.0

Claim protocol: [IMPLEMENTED] · [SIMULATED] · [PARTIALLY VERIFIED] · [FUTURE / NOT IMPLEMENTED]

Ground truth: Every claim sourced from direct file inspection of the repo ZIP. Nothing inferred from PRD text alone.

Section 0 — Repo Grounding Checklist

Files inspected for this document

Area	Files / Paths Inspected	Status
Repo ZIP	metasignal_platform-main.zip extracted	✓
v1 PDF	MetaSignal_Interview_Defense_Docu ment.pdf	✓
Top-level structure	README.md, pyproject.toml, alembic.ini	✓
Source — DB	src/metasignal/db/models.py, src/metasignal/db/session.py	✓
Source — API	src/metasignal/api/app.py	✓
Source — Streaming	src/metasignal/streaming/streaming_p rd_v1.py	✓
Alembic migrations	alembic/versions/61738c5dba04_crea te_core_metasignal_tables.py	✓
Alembic migrations	alembic/versions/de8aedf29d09_add_ experiment_assignments.py	✓
Scripts — orchestrators	run_full_demo_v0.py, run_metasignal_prd_complete_v1.py	✓
Scripts — core	seed_core.py, compute_metrics_v0.py, generate_assignments_v0.py	✓
Scripts — experiment	run_srm_check_v0.py, run_cuped_v0.py, run_guardrail_decision_v0.py	✓

Area	Files / Paths Inspected	Status
Scripts — validation	run_cuped_aa_validation_v0.py, run_prd_core_validation_suite_v1.py	✓
Scripts — anomaly	run_anomaly_detector_v0.py (anomaly backtest is inside prd_core_validation_suite_v1)	✓
Scripts — operational	seed_operational_history_v1.py (partial — large file)	✓
Scripts — post-eval	run_post_launch_validation_v0.py, run_golden_scenarios_v0.py	✓
Scripts — streaming	run_streaming_demo_v0.py, show_streaming_demo.py, validate_streaming_prd_v1.py	✓
Scripts — API tests	smoke_test_api_v0.py, test_override_enforcement_v1.py	✓
Scripts — exports	export_resume_signal_summary_v0.p y, validate_evidence_artifacts_v0.py	✓
Scripts — misc	detect_metric_conflicts_v0.py, run_failure_injection_v0.py	✓
GitHub Actions	.github/workflows/metasignal-validatio n.yml	✓
Docs	METASIGNAL_ARCHITECTURE.md, METASIGNAL_DEMO_NARRATIVE. md	✓
Docs	METASIGNAL_INTERVIEW_DEFEN SE.md, METASIGNAL_PRD_COMPL ETION_MAP.md	✓
Docs	METASIGNAL_STREAMING_EXTEN SION.md	✓
outputs/evidence	All 21 JSON files listed and key ones fully read	✓
outputs/validation	All 8 JSON files listed; cuped_aa, cuped_edge_case, streaming_prd_v1, override read fully	✓
outputs/streaming	14+ JSON files listed; key ones (srm_alerts, dlq, recon, scenario_report) read	✓
outputs/reports	All 9 files listed; prd_completion_report_v1, resume_signal fully read	✓

Files not found or not confirmed

File	Status	Notes
run_anomaly_backtest_v0.py	NOT a separate file	Anomaly backtest logic is inside <code>run_prd_core_validation_suite_v1.py</code> as <code>run_anomaly_backtest_v1()</code>
dlq_events.parquet	Conditionally present	Written if pandas + pyarrow available; fallback JSON written otherwise
data/interim/events_standardized.parquet	Not inspectable as text	Presence confirmed by scripts that require it; 2,756,101 rows confirmed by DQ report
scripts/seed_operational_history_v1.py full text	Partially read (first 150 lines)	Key output (<code>operational_history_60_day_report.json</code>) fully read — 13 scenarios, 7 metrics, 60 health reports confirmed
src/metasignal/db/session.py	Not read	Referenced by all scripts; standard SQLAlchemy SessionLocal pattern

Section 1 — Truth Boundary and Project Identity

The one sentence you must say first in every interview

"MetaSignal is a solo-built, non-production, production-simulated experimentation intelligence platform. Every claim I make can be traced to a JSON artifact or a database row in the repo."

The four-level truth hierarchy

MetaSignal evidence exists at four distinct levels. Interviewers who push on "is this real?" are usually distinguishing between levels 1 and 4. Know these cold.

Level 1 — Core DB state [IMPLEMENTED]

9 tables created by Alembic migrations. 3 metric definitions, 20,000 assignments, 2 evaluations, 2 decision logs, 51 pipeline runs, 417 metric results, and 30+ failed quality checks. These rows exist in a real PostgreSQL database and are queryable. Created by `seed_core.py` + the v0 demo scripts.

Level 2 — Operational history seeder [SIMULATED]

`seed_operational_history_v1.py` creates 4 support tables (not in Alembic) and seeds 60 days of simulated operational history: 7 metric definitions (3 core + 4 PRD-level), 4 conflicts, 4 backfill jobs, 60 system health reports, 2 anomaly alerts, and 13 simulation scenarios (SIM-001 through SIM-013). This is deliberate simulation of production operational patterns — the logic is real but the events are scripted.

Level 3 — Core validation scripts [IMPLEMENTED]

Scripts in `run_prd_core_validation_suite_v1.py` implement and validate CUPED edge cases, 12 golden scenarios, peeking/novelty/Simpson's causal traps, and the anomaly backtest. These run as pure Python (no DB needed for all checks) and produce artifacts with `status: pass`.

Level 4 — Artifact-level evidence [IMPLEMENTED]

The `outputs/` directories contain 50+ JSON files. Each artifact has an `artifact` field, key numeric fields, and an `evidence_statement`. These are the interviewer-facing proof layer. They are deterministic — if the repo is rerun, the same numbers appear.

What "production-simulated" means precisely

From `docs/METASIGNAL_PRD_COMPLETION_MAP.md` (verbatim):

"MetaSignal Core PRD and Streaming Extension PRD are implemented at repo-evidence level. This means the repository now contains executable scripts, validation reports, and committed artifacts demonstrating the PRD behaviours. This does NOT mean real production deployment, real users, real traffic, or enterprise-grade infrastructure."

What is real:

- The PostgreSQL database is real (spun up locally)
- The RetailRocket dataset is a real public dataset with 2,756,101 rows
- The CUPED algorithm is a real implementation (Cov/Var regression, z-test, A/A validation)
- The SRM chi-square test is real
- The anomaly detection algorithm is real (DOW-adjusted z-score)
- The FastAPI layer is real and smoke-tested
- The GitHub Actions CI is real and runs on push
- The Alembic migrations are real and auto-generated from SQLAlchemy models

What is synthetic / simulated:

- The A/B treatment effect (255 users deterministically injected)
- The refund guardrail breach (3.1% hardcoded)
- The streaming events (asyncio simulation, no Kafka)
- The anomaly backtest labels (20 events hand-crafted)
- The peeking/novelty/Simpson's data (hardcoded numbers in `prd_core_validation_suite_v1.py`)
- The operational history scenarios (scripted day-by-day events)
- The causal trap evaluation history (days 3/7/10 p-values hardcoded)

What does not exist:

- Real users or analysts making decisions
- Production traffic or live deployment
- Kafka, Flink, Spark, or real streaming infrastructure
- A dashboard or frontend
- Real guardrail breach data (no `return_initiated` events in RetailRocket)

Spoken openers

One-line truth:

"MetaSignal is a solo-built, production-simulated experimentation platform with real DB rows, real statistical code, and real JSON evidence — but no live traffic and no real users."

Recruiter pitch:

"I built MetaSignal to show I understand how experimentation infrastructure actually works at platform companies — metric governance, quality gates, CUPED, guardrail-first decisions, anomaly detection, and a streaming early-warning layer."

Technical pitch:

"MetaSignal is a batch-authoritative experimentation platform with versioned metric registry, denominator conflict detection, deterministic assignment, SRM checks, CUPED with A/A validation, guardrail-first decisioning, right-censoring, DOW-adjusted anomaly detection, causal trap flags, and an asyncio-simulated streaming layer — all producing

machine-readable evidence artifacts."

30-second spoken opener:

"MetaSignal is a production-simulated experimentation platform I built from scratch. I used the RetailRocket public dataset — 2.75 million e-commerce events — loaded it into PostgreSQL via SQLAlchemy, and built the full experiment lifecycle on top: metric registry with denominator governance, data quality gates, deterministic user assignment, CUPED readout, guardrail-first decisioning, and a streaming early-warning layer. It's not a production system and the A/B effect is synthetic, but every claim traces to a JSON artifact or a DB row."

60-second spoken opener:

"MetaSignal is a solo-built, production-simulated experimentation intelligence platform. Let me be upfront: it's not a live system — there are no real users, no Kafka, no production traffic. But the data is real — RetailRocket, 2.75 million events, standardized into a parquet file and loaded into a real PostgreSQL database via Alembic migrations. On top of that I built: a metric registry that forces explicit numerator and denominator SQL; a denominator conflict detector; data quality gates that block metric computation if data is bad; deterministic SHA-256 user assignment; SRM checks before readout; CUPED variance reduction with a 1000-run A/A validation suite; guardrail-first decisioning where a positive primary metric gets held because a refund guardrail breaches tolerance; right-censoring on delayed metrics; anomaly detection with DOW adjustment; peeking/novelty/Simpson's paradox detection; and an async-based streaming layer that simulates SRM drift, DLQ events, consumer lag spikes, and stream-batch reconciliation. Every behavior produces a JSON artifact. The run command is a single line."

2-minute spoken opener:

"Let me start with what MetaSignal is and isn't, because I want to be precise. It's not a production system. It's solo-built and production-simulated — meaning I deliberately engineered it to handle realistic operational failure modes, produce machine-readable evidence, and demonstrate the kind of systems thinking I'd apply to a real platform. The data is real: RetailRocket, a public e-commerce dataset with 2.75 million events. I standardized it to a parquet file with columns for user_id, event_date, event_type, and transaction_id, then loaded it into PostgreSQL via SQLAlchemy ORM with Alembic migrations.

>

The A/B treatment effect is synthetic — I deterministically injected about 255 treatment users to produce a non-null readout, because RetailRocket doesn't have a real A/B experiment. That's documented in the code and in the evidence artifact. Similarly, the refund guardrail breach is hardcoded at 3.1% because the dataset doesn't have real return events. These aren't embarrassments — they're documented design decisions.

>

What's real is the statistical infrastructure. The CUPED implementation computes theta via Cov/Var and produces an adjusted z-test. I validated it on 1,000 synthetic A/A runs — false positive rate is 5.5%, the KS test against Uniform(0,1) passes, mean variance reduction is 26.5% on synthetic data with controlled 0.6 correlation. In the actual RetailRocket experiment, variance reduction is only 0.44% because pre-event count has low predictive power for binary purchase conversion — that's honest and I can explain why.

>

The guardrail system demonstrates the guardrail-first principle: the primary metric shows 420% relative lift at $p=5e-44$, and the system still says HOLD because the refund guardrail breaches its 2% tolerance. The API enforces that if a human reviewer overrides the system recommendation, they must provide a written reason — tested and verified against HTTP 422.

>

The streaming layer is async simulation — no Kafka. But it implements 15 classes, 10 failure scenarios (SRM drift, iOS event gaps, late events, duplicates, DLQ poisoning, consumer lag spikes, provisional anomalies, stream-batch mismatch, type mismatch, replay recovery), and all 43 validation checks pass. The hard rule is: streaming is provisional only — it triggers investigation, never final decisions.

>

The whole thing runs from one command. The run produces 50+ JSON evidence artifacts and a set of DB rows. Everything I say about this project can be verified."

Section 2 — Executive Positioning

Why this project exists and what it proves

Most data science portfolios show models. MetaSignal shows infrastructure. The gap it closes is between knowing statistics and knowing how statistics gets embedded in systems that real companies actually trust.

The capabilities MetaSignal demonstrates, in order of interview value:

1. Metrics intelligence — not just dashboards [IMPLEMENTED]

Metric definitions in MetaSignal store numerator SQL, denominator SQL, denominator type, grain, entity type, direction, owner, version, config hash, and guardrail parameters. This is not a `value` column in a dashboard table. It is a governed, versioned, conflict-detectable definition. Teams can't accidentally use different denominators for the same concept without the system catching it.

2. Denominator governance [IMPLEMENTED]

The conflict detector groups metrics by "family" (inferred from name) and flags any family where active metrics use different `denominator_type`, `grain`, or `entity_type`. Two conversion rate metrics are live in the DB with different denominators: `event_count` vs `distinct_users`. Same name family, different denominators → conflict detected, artifact written.

3. Data quality as a gate, not a check [IMPLEMENTED]

Quality checks use `severity=blocking`. If a blocking check fails, metric computation is prevented entirely. Three failure injection scenarios (schema drift, invalid event type, null spike) all produce `status: blocked` and `expected_behavior_met: true`. The system produces no metric when data is untrustworthy.

4. Experiment decisioning with audit trail [IMPLEMENTED]

Every experiment flows through: assignment → SRM → CUPED evaluation → guardrail check → decision log. The decision log records `system_recommendation`, `final_decision`, `reviewer_name`, and requires `override_reason` when they differ. This is an audit trail, not just a result.

5. Guardrail-first, not primary-first [IMPLEMENTED]

The experiment in the repo has $p=5e-44$ and 420% relative lift. The system recommendation is HOLD. This is the single most important behavioral demonstration. Guardrails are not optional post-hoc checks — they block ship decisions regardless of primary metric strength.

6. Right-censoring and delayed guardrails [IMPLEMENTED]

The refund guardrail's maturity window is 30 days. The readout happened at day 14. The system marks it `right_censoring_flag: true`, `is_mature: false`, and `status: immature_but_breached`. This prevents premature final decisions based on incomplete guardrail readings.

7. Batch-authoritative with streaming early warning [IMPLEMENTED / SIMULATED]

The streaming layer is explicitly provisional. From `docs/METASIGNAL_STREAMING_EXTENSION.md`: "Streaming alerts may trigger investigation or pause-for-review, but never final experiment decisions." Batch remains authoritative. Reconciliation tracks the delta between streaming provisional values and batch authoritative values.

8. Reproducibility and audit via config hash [IMPLEMENTED]

Each `MetricDefinition` has a `config_hash` (SHA-256 of definition content). `MetricResult` rows store `metric_version`. If a definition changes, the hash changes, a new version is created, and historical results remain linked to the old definition. You

can't retroactively corrupt historical metric comparisons.

9. Anomaly detection with DOW adjustment [IMPLEMENTED]

Anomaly detection uses same-day-of-week rolling baselines because e-commerce metrics have strong weekly seasonality. Without DOW adjustment, normal Monday patterns would fire on weekends and vice versa. The backtest shows DOW adjustment suppresses 3 false positives that raw rolling z-score would fire on.

10. Statistical discipline — not just significance [IMPLEMENTED]

MetaSignal flags peeking (significance flipped across 3 interim reads), novelty effect (week-1 lift is 7x week-2 lift), and Simpson's paradox (overall +3.1% but IN=-1.2%, iOS=-1.8%). These are not sidebar features — they're stored as Boolean columns in ExperimentEvaluation and surfaced via API.

Why this is stronger than a notebook project

A notebook project:

- Shows statistical knowledge in isolation
- Has no governance layer (anyone can rename a metric)
- Has no quality gates (bad data silently produces wrong metrics)
- Has no audit trail (decisions are not logged)
- Has no reproducibility guarantee (notebook state depends on run order)
- Has no streaming considerations
- Cannot be extended by a second person without breaking assumptions

MetaSignal addresses every one of these. It is not a better notebook — it is a different category of artifact.

Section 3 — End-to-End Architecture

Text diagram: Core / Batch Architecture

[illegible]


```

TRANSPORT (NOT Kafka) ■ ■ AsyncSimulatorEventSource — generates events via asyncio ■ ■ 12 windows × 500
base events = ~6,000 raw events ■ ■ RNG_SEED=202606 for reproducibility ■ ■ Failure scenarios injected by
window_id conditions in source ■ ■ Truth: [SIMULATED] asyncio. [IMPLEMENTED] interface design. ■
yields events ▼
INGESTION ■ ■ AsyncEventQueue — in-memory buffer, awaitable put/drain ■ ■ StreamEventConsumer —
orchestrates source → queue → validate ■
VALIDATION (StreamValidator) ■ ■ 13 required fields checked. Invalid variant → DLQ. Non-numeric value →
DLQ ■ ■ Duplicate event_id → separate duplicates list ■ ■ Late event (>15 min before window watermark) →
late_events list ■ ■ DLQ writer: writes dlq_events.json + dlq_events.parquet (if pyarrow avail) ■ ■ ■
Error codes: MISSING_REQUIRED_FIELD · INVALID_VARIANT · SCHEMA_TYPE_MISMATCH ■
SRMDetector ■ ■ Traffic ■ ■ Instrumentation ■ ■ Rolling 6-win ■ ■ Allocation ■ ■
Health Monitor ■ ■ SRM check ■ ■ Monitor ■ ■ Platform event ■ ■ >5pp → invest ■ ■ Per-window ■ ■ drop
<30% base ■ ■ >2pp → warn ■ ■ share snapshot ■ ■ → critical/inv ■ ■
ConsumerLagMonitor ■ ■
Per-window P50/P95/P99/max lag ■ ■ P95 > 300s → alert_level=investigate ■ ■ Window 7: lag injected at
1200-1500s → fires ■ ■
RealtimeMetricSnapshotter ■ ■
Per-window purchase rate by variant ■ ■ ALL snapshots labeled provisional_label=true ■ ■
ProvisionalAnomalyDetector ■ ■
EWMA-style: fires if metric > 2× rolling 4-window ■ ■ baseline. lag_suppressed if P95 > 300s ■ ■
StreamQualityChecker +
StreamBatchReconciler ■ ■ Per-window: min_valid_events, malformed_rate, ■ ■ duplicate_rate,
late_event_rate, lag_p95 ■ ■ Reconciliation: streaming ≈ batch × 0.94 (simulated) ■ ■
reconciliation_status: minor_delta ■ ■ batch_remains_authoritative=true ■ ■
ReplayManager ■ ■ replay_id:
replay_20260401_1100 ■ ■ triggered_by: consumer_recovery_after_two_hour_gap ■ ■ batch_truth_unchanged=true

```

Section 4 — Database / Schema Defense

The two-layer schema reality

Layer A — Alembic-migrated tables (9 tables, 2 migrations):

These are authoritative. They are version-controlled, auto-generated, and fully reversible.

Layer B — Support tables (4 tables, raw SQL in seed_operational_history_v1.py):

Created by CREATE TABLE IF NOT EXISTS raw SQL. Not in Alembic. They are dropped and recreated on each seed. Exist for operational history simulation only.

Table-by-table defense

metric_definitions — Core, Alembic-migrated

Where defined: src/metasignal/db/models.py → MetricDefinition. Migration: 61738c5dba04.

Key columns: `id` (UUID PK), `name` (String 128), `display_name`, `description`, `numerator_sql` (Text), `denominator_sql` (Text), `denominator_type` (String 64), `filter_sql`, `grain` (String 32), `entity_type` (String 32), `direction` (String 32), `is_guardrail` (Boolean), `guardrail_tolerance_pct` (Float), `guardrail_window_days` (Int), `owner` (String 128), `version` (Int), `is_active` (Boolean), `deprecated_at`, `parent_version_id` (UUID, self-FK), `change_notes` (Text), `config_hash` (String 128), `created_at`.

Core DB state (3 rows from `seed_core.py`):

Name	Denominator Type	Is Guardrail	Version
purchase_conversion_rate_user	distinct_users	false	1
purchase_conversion_rate_event	event_count	false	1
refund_rate_guardrail	purchase_events	true	1

PRD-level state (7 rows after `seed_operational_history_v1.py`): The seeder adds 4 more metrics for PRD scenario coverage (including DAU v1/v2, session-level metrics). Only 3 are in the core Alembic baseline. If asked "how many metrics?", the answer depends on which run level you're at: 3 in core seed, 7 in operational history scenario.

Why it exists: Forces teams to declare their metric definition explicitly. The `config_hash` ensures immutability — if the SQL changes, it's a new version. `Parent_version_id` creates version lineage.

Interview value: This is the anchor for denominator governance. When asked "how do you prevent metric inconsistency?", point to this table.

Limitations: Only 3 core metrics. A real platform would have hundreds. The `config_hash` is SHA-256 of a simple string, not a cryptographic guarantee of SQL equivalence.

metric_results — Core, Alembic-migrated

Where defined: `MetricResult` model. Migration: 61738c5dba04.

Key columns: `id` (UUID PK), `metric_id` (FK → `metric_definitions`), `metric_version` (Int), `computation_date` (Date), `grain_period` (String 64), `numerator_value` (Float), `denominator_value` (Float), `metric_value` (Float), `segment_key` (String 256), `run_id` (FK → `pipeline_run_log`), `quality_passed` (Boolean), `late_arrival_flag` (Boolean), `created_at`.

Unique constraint: (`metric_id`, `metric_version`, `grain_period`, `segment_key`) — prevents duplicate results for the same definition version, period, and segment.

Live state: 417 rows — daily results for all 3 metrics over the RetailRocket date range.

Why it exists: Results are immutable per version. If a metric definition changes, the new version's results don't overwrite the old version's. Historical comparisons remain valid.

Interview value: The unique constraint demonstrates thinking about data integrity, not just computation.

pipeline_run_log — Core, Alembic-migrated

Key columns: `id` (UUID PK), `job_type` (String 64), `run_date` (Date), `started_at`, `completed_at`, `status` (String 64), `rows_processed` (Int), `rows_expected` (Int), `quality_checks_passed` (Int), `quality_checks_failed` (Int), `error_message` (Text), `config_hash`, `retry_count` (Int), `triggered_by`

(String 64).

Live state: 51 rows (from 60-day operational history seed + core demo runs).

Why it exists: Every metric computation and data quality run is traceable. When an anomaly fires, you can look up the pipeline run that produced the underlying metric to check for quality failures.

data_quality_checks — Core, Alembic-migrated

Key columns: id, run_id (FK → pipeline_run_log), check_name, entity, check_date, expected_value (Float), actual_value (Float), threshold (Float), passed (Boolean), severity (String 32), check_details (JSONB).

Live state: 30 failed checks (from failure injection). More passing checks from clean runs.

Why it exists: Individual check details are stored as JSONB — the expected vs actual mismatch is queryable. You can reconstruct exactly what was wrong with a given run.

Limitations: check_details is JSONB, which makes it flexible but not schema-enforced. A real system might use more structured check result tables.

simulation_events — Core, Alembic-migrated

Key columns: id, scenario_id (String 64, unique), scenario_name, scenario_type, simulation_day (Int), event_date (Date), injected (Boolean), injection_payload (JSONB), expected_system_behavior (Text).

Live state: 5 rows from seed_core.py. 13 additional rows from seed_operational_history_v1.py (SIM-001..SIM-013, different scenario_id prefix).

Why it exists: Documents the intended behavior for each failure scenario. The expected_system_behavior field is human-readable test documentation embedded in the DB.

experiments — Core, Alembic-migrated

Key columns: id, experiment_key (String 64, unique), name, hypothesis (Text), start_date, end_date, status, primary_metric_id (FK → metric_definitions), guardrail_metric_configs (JSONB), expected_mde (Float), expected_duration_days (Int), alpha (Float), power (Float), use_cuped (Boolean), assignment_method (String 64), treatment_split_pct (Float), notes (Text), created_by.

Live state: 1 experiment — EXP-CHECKOUT-FRICTION-001, 2015-05-10 to 2015-05-24, use_cuped=true, assignment_method=deterministic_hash, treatment_split_pct=0.5.

experiment_assignments — Core, Alembic-migrated (migration 2)

Key columns: id, experiment_id (FK), user_id (String 128), variant (String 32), assignment_hash (String 128), assignment_bucket (Float), assignment_method (String 64), assigned_at.

Unique constraint: (experiment_id, user_id) — one assignment per user per experiment.

Live state: 20,000 rows. 10,094 treatment, 9,906 control.

Interview value: The unique constraint plus deterministic hash guarantees idempotent, consistent assignment. Same user always gets the same variant. assignment_hash is the full SHA-256 hex for auditability.

experiment_evaluations — Core, Alembic-migrated

Key columns: id, experiment_id (FK), evaluated_at, evaluation_day (Int), sample_size_treatment (Int), sample_size_control (Int), primary_metric_lift (Float), primary_metric_ci_lower, primary_metric_ci_upper, p_value, is_significant (Boolean), cuped_applied (Boolean), cuped_theta (Float), variance_reduction_pct (Float), cuped_edge_case (String 64), cuped_fallback (String 64), guardrail_results (JSONB), guardrails_cleared (Boolean), peeking_warning (Boolean), simpsons_flag (Boolean), segment_analysis (JSONB), novelty_flag (Boolean), decision_recommendation (String 64), recommendation_reason (Text), evaluator_version (String 64), is_aa_test (Boolean).

Live state: 2 evaluation rows, both with cuped_applied=true, both with decision_recommendation=HOLD, evaluator_version=cuped_v0.

Why it exists: The evaluation row is the single source of truth for what the system computed. Every flag — peeking, novelty, simpsons, cuped edge cases — is a first-class column, not a side-artifact.

decision_log — Core, Alembic-migrated

Key columns: id, experiment_id (FK), evaluation_id (FK), system_recommendation (String 64), final_decision (String 64), reviewer_name (String 128), override_reason (Text, nullable), decision_notes.

Live state: 2 rows. Both final_decision=HOLD. override_reason=null (system recommendation accepted).

Why it exists: Immutable audit record. You cannot change a decision after it's made without creating a new row. override_reason must be non-null when human decision differs from system recommendation (enforced at API level, not DB constraint level).

Limitation: The DB does not enforce override_reason as a constraint. Enforcement is in the API layer (POST /decisions/validate-override). A sophisticated interviewer may note this. Safe answer: "The API is the enforcement boundary. A DB constraint could be added — I chose API-level enforcement to keep the constraint logic testable without a DB migration."

Support tables (created by seed_operational_history_v1.py, NOT in Alembic)

metric_conflicts

Columns: id (UUID PK), metric_a_id (UUID), metric_b_id (UUID), conflict_type (Text), value_divergence_pct (Float), sample_period (Text), detected_at (TIMESTAMP TZ), resolved (Boolean), resolution_notes (Text). Created with CREATE TABLE IF NOT EXISTS. 4 conflict rows in live state.

backfill_jobs

Columns: id (UUID PK), metric_id (UUID), trigger_reason (Text), affected_dates (Text), triggered_by_run_id (UUID), status (Text), started_at, completed_at, rows_recomputed (Int), notes (Text). 4 backfill rows.

anomaly_alerts

Columns: id (UUID PK), metric_id (UUID), detection_date (Date), metric_value (Float), expected_value (Float), z_score (Float), ewma_deviation (Float), dow_adjusted (Boolean), severity (Text), detection_method (Text), attribution (Text), acknowledged (Boolean), acknowledged_by (Text), acknowledged_at, false_positive (Boolean), feedback_notes (Text), injected_anomaly (Boolean). 2 alert rows.

system_health_reports

Columns: id (UUID PK), report_date (Date), pipeline_success_rate (Float), metrics_fresh_count (Int), metrics_stale_count (Int), anomaly_alerts_fired (Int), anomaly_alerts_acknowledged (Int), anomaly_false_positive_rate (Float), experiment_evaluations_completed (Int), decisions_made (Int), decisions_overridden (Int), decision_override_rate (Float), post_launch_validations_due (Int), post_launch_validations_completed (Int), post_launch_validation_failures (Int), notes (Text). 60 rows (one per seeded day).

Why these are not in Alembic: These tables exist to demonstrate PRD-level operational patterns, not as part of the core system. They are seeded and recreated by the operational history script, not maintained as durable schema.

Interview answer when asked: "Four support tables are created by the operational history seeder using raw SQL, not Alembic. They exist to demonstrate 60-day operational patterns — backfill tracking, anomaly alert lifecycle, daily health reports. The core 9 tables that matter for the experiment lifecycle are fully Alembic-migrated."

Section 5 — Script-by-Script Defense

Orchestrator scripts

Script	Purpose	Reads	Writes	Key artifact	Truth
run_full_demo_v0.py	Runs all 24 v0 scripts in sequence. Exit on first failure.	—	—	None (orchestrator)	[IMPLEMENTED]
run_metasignal_prd_complete_v1.py	Runs v1 completion pipeline: seed_history → prd_core_validation → override_enforcement → streaming_demo → streaming_validation. Checks all 23 required artifacts. Writes PRD completion report.	—	outputs/reports/metasignal_prd_completion_report_v1.json	prd_completion_report_v1	[IMPLEMENTED]

run_full_demo_v0.py — how to run:

```
PYTHONPATH=. python3 scripts/run_full_demo_v0.py
```

Success: all 24 scripts exit 0. Outputs printed to stdout. Artifacts written to outputs/.

run_metasignal_prd_complete_v1.py — how to run:

```
PYTHONPATH=. python3 scripts/run_metasignal_prd_complete_v1.py
```

Success: status: pass, 13 core PRD items confirmed, 21 streaming items confirmed.

Seeder scripts

Script	Purpose	Reads	Writes	Key artifact	Truth
seed_core.py	Creates 3 metric definitions, EXP-CHECKOUT-FRICTION-001 experiment, 5 simulation events. Deletes prior seed data first (idempotent).	DB	DB rows	None (seeds DB)	[IMPLEMENTED]
seed_operational_history_v1.py	Creates 4 support tables. Seeds 60-day operational history: 7 metrics, 4 conflicts, 4 backfill jobs, 60 health reports, 13 simulation scenarios, 2 anomaly alerts.	DB	DB rows + support tables	outputs/evidence/operational_history_60_day_report.json	[SIMULATED]

seed_core.py — what to say: "Idempotent seeder that creates the 3 core metric definitions and the one experiment. Deletes prior data before inserting. Safe to rerun."

seed_operational_history_v1.py — what to say: "Simulates 60 days of operational patterns including 13 labeled failure scenarios: schema drift, null spike, late data, double-counting, iOS outage, DOW false positive, metric versioning, experiment lifecycle, delayed guardrail, human override, and clean ship. These are scripted events — they're simulation, not production logs."

Data pipeline scripts

Script	Purpose	Reads	Writes	Key artifact	Truth
generate_events.py	Standardizes raw RetailRocket data into events_standardized.parquet	Raw RetailRocket CSV/parquet	data/interim/events_standardized.parquet	—	[IMPLEMENTED]
run_data_quality_v0.py	5 quality checks on events parquet. Blocking severity.	events parquet, DB	DB (PipelineRunLog, DataQualityCheck rows)	data_quality_service_report.json	[IMPLEMENTED]
run_failure_injection_v0.py	3 bad-data scenarios. Each should produce status=blocked.	events parquet, DB	DB (failed check rows)	failure_injection_report.json	[SIMULATED]

Script	Purpose	Reads	Writes	Key artifact	Truth
compute_metrics_v0.py	Computes 3 metrics daily from events parquet. Writes 417 MetricResult rows.	events parquet, DB	DB (MetricResult rows, PipelineRunLog)	metric_results_summary.json	[IMPLEMENTED]
export_metric_results_summary.py	Exports metric result summary from DB to JSON.	DB	outputs/evidence/metric_results_summary.json	metric_results_summary	[IMPLEMENTED]

Metric governance scripts

Script	Purpose	Reads	Writes	Key artifact	Truth
detect_metric_conflicts_v0.py	Groups active metrics by family. Detects denominator/grain/entity conflicts.	DB	outputs/evidence/metric_conflict_report.json	metric_conflict_report	[IMPLEMENTED]
export_core_artifacts.py	Exports DB state to JSON evidence files.	DB	Multiple evidence JSONs	core_evidence_manifest	[IMPLEMENTED]

Experiment scripts

Script	Purpose	Reads	Writes	Key artifact	Truth
generate_assignments_v0.py	SHA-256 deterministic assignment for 20,000 users. 50/50 split.	events parquet, DB	DB (20,000 ExperimentAssignment rows)	experiment_assignment_balance.json	[IMPLEMENTED]
run_srm_check_v0.py	Chi-sq goodness-of-fit on assignment counts. Chi-sq=1.77, p=0.18, no SRM.	DB	outputs/evidence/srm_check_report.json	srm_check_report	[IMPLEMENTED]

Script	Purpose	Reads	Writes	Key artifact	Truth
run_cuped_v0.py	Full CUPED pipeline: load events, compute pre-period covariate, inject 255 synthetic uplift users, compute θ , adjust outcomes, z-test. Writes ExperimentEvaluation row.	events parquet, DB	DB (ExperimentEvaluation), cuped_experiment_readout.json	cuped_experiment_readout	[IMPLEMENTED] computation + [SIMULATED] uplift
run_guardrail_decision_v0.py	Loads CUPED eval. Evaluates guardrail (3.1% vs 2.0%). HOLD. Writes DecisionLog.	DB	DB (DecisionLog, updated ExperimentEvaluation), guardrail_decision_report.json	guardrail_decision_report	[IMPLEMENTED] logic + [SIMULATED] breach
run_post_launch_validation_v0.py	Right-censoring check. Refund guardrail day-14 = immature (maturity window 30 days).	DB	outputs/evidence/post_launch_validation_report.json	post_launch_validation_report	[IMPLEMENTED] logic + [SIMULATED] numbers

Validation scripts

Script	Purpose	Reads	Writes	Key artifact	Truth
run_cuped_aa_validation_v0.py	1,000 synthetic A/A runs. FPR=5.5%, KS test pass, mean VR=26.5%.	— (synthetic numpy)	outputs/validation/cuped_aa_validation_report.json	cuped_aa_validation_report	[IMPLEMENTED]
run_prd_core_validation_suite_v1.py	Runs: CUPED edge cases (5), golden scenarios v1 (12), causal trap detectors (peeking/novelty/Simpson's), anomaly backtest (20 events).	— (mostly in-memory)	4 JSON artifacts + prd_core_validation_suite_v1_summary	Multiple	[IMPLEMENTED] logic + [SIMULATED] injected values

Script	Purpose	Reads	Writes	Key artifact	Truth
test_override_enforcement_v1.py	Uses FastAPI TestClient. Sends override POST without reason → expects 422. With reason → expects 200.	FastAPI app	outputs/validation/override_reason_enforcement_report.json	override_reason_enforcement	[IMPLEMENTED]
validate_evidence_artifacts_v0.py	Checks that all expected evidence artifact files exist and are non-empty valid JSON.	File system	outputs/validation/evidence_artifact_validation_report.json	evidence_artifact_validation	[IMPLEMENTED]
smoke_test_api_v0.py	FastAPI TestClient hits 8 GET endpoints. All should return 200.	FastAPI app, DB, file system	outputs/validation/api_smoke_test_report.json	api_smoke_test_report	[IMPLEMENTED]

Anomaly / reliability scripts

Script	Purpose	Reads	Writes	Key artifact	Truth
run_anomaly_detector_v0.py	DOW-adjusted z-score on purchase_conversion_rate_user timeseries. Injects synthetic anomaly at day 45 (4x baseline). Detected.	DB (MetricResult rows)	outputs/evidence/anomaly_detection_report.json	anomaly_detection_report	[IMPLEMENTED] algorithm + [SIMULATED] injection
run_golden_scenarios_v0.py	8 behavioral assertions against live DB state. All pass.	DB	outputs/evidence/golden_scenario_suite_report.json	golden_scenario_suite_report	[IMPLEMENTED]

Streaming scripts

Script	Purpose	Reads	Writes	Key artifact	Truth
run_streaming_demo_v0.py	Calls run_sync() from streaming_prd_v1. Same as show_streaming_demo.py.	—	All outputs/streaming/*.json	All streaming artifacts	[IMPLEMENTED] logic + [SIMULATED] events
show_streaming_demo.py	Wrapper for run_sync(). Prints summary to stdout.	—	— (reuses run_streaming outputs)	—	[IMPLEMENTED]
validate_streaming_prd_v1.py	43 checks: 15 class existence + 14 artifact existence + 13 content checks + 1 parquet/fallback. All 43 pass.	streaming module + file system	outputs/validation/streaming_prd_v1_validation_report.json	streaming_prd_v1_validation	[IMPLEMENTED]
validate_streaming_extension_v0.py	Earlier streaming validation (before v1).	streaming artifacts	outputs/validation/streaming_extension_validation_report.json	streaming_extension_validation	[IMPLEMENTED]

Export / report scripts

Script	Purpose	Key artifact	Notes
export_resume_signal_summary_v0.py	Writes resume bullet + 10 interview signals	metasignal_resume_signal_summary.json + .md	Good pre-interview reference
export_demo_narrative_v0.py	Writes 10-step demo story	metasignal_demo_narrative.json	Useful for demo prep
export_architecture_summary_v0.py	Writes architecture layer summary	metasignal_architecture_summary.json	Mirrors docs/
export_interview_defense_v0.py	Writes short interview Q&A	metasignal_interview_defense.json	Superseded by this document
export_core_artifacts.py	Exports DB state to evidence JSON files	core_evidence_manifest.json	Runs after DB is populated
export_final_completion_report_v0.py	Writes final completion summary	metasignal_final_completion_report.json	High-level status

Script	Purpose	Key artifact	Notes
export_streaming_extension_summary_v0.py	Writes streaming capability summary	metasignal_streaming_extension_summary.json	Mirrors docs/
show_core_report.py	Prints core status summary to stdout	—	Terminal readout only
export_core_completion_report_v0.py	Writes core completion report	metasignal_core_completion_report.json	Earlier v0 version

Section 6 — Artifact-by-Artifact Defense

How to use this section

For each artifact, the format is: **what it proves**, **what it does NOT prove**, **exact safe wording**, **numbers to cite**, **unsafe wording to avoid**.

outputs/evidence/metric_registry.json

Produces: export_core_artifacts.py / DB state

Key fields: 3 objects, each with id, name, numerator_sql, denominator_sql, denominator_type, grain, entity_type, is_guardrail, version, config_hash

Key numbers: 3 metric definitions. refund_rate_guardrail has guardrail_tolerance_pct: 2.0

Proves: Metric governance schema exists and is populated. Denominator SQL is explicit per definition.

Does NOT prove: That these metrics are used by a real team or produce accurate business numbers.

Safe wording: "The registry has 3 active metrics with explicit numerator and denominator SQL, grain, entity type, and config hash."

Unsafe wording: "We track 3 production metrics."

outputs/evidence/metric_conflict_report.json

Produces: detect_metric_conflicts_v0.py

Key fields: conflict_count, conflicts[] each with conflict_type, denominator_types, affected_metrics

Key numbers: 1 conflict in core state (conversion_rate family), 4 conflicts in PRD operational history state

Proves: The detector correctly identifies active metrics with mismatched denominator types in the same family.

Does NOT prove: That the conflict detection covers all real-world conflict patterns.

Safe wording: "The conflict detector found that two active conversion rate metrics use different denominators — event_count vs distinct_users — and flagged this as a semantic conflict."

Unsafe wording: "We detect all metric conflicts."

outputs/evidence/data_quality_service_report.json

Produces: run_data_quality_v0.py

Key fields: rows_processed: 2756101, checks_passed: 5, checks_failed: 0, status: success

Proves: 5 quality checks pass on the RetailRocket parquet. Pipeline run logged in DB.

Does NOT prove: Comprehensive data quality coverage. Only 5 checks defined.

Safe wording: "The data quality gate ran 5 checks on 2.75 million rows and passed all of them before metric computation."

Unsafe wording: "Data quality is fully covered."

outputs/evidence/failure_injection_report.json

Produces: run_failure_injection_v0.py

Key numbers: 3 scenarios, 3 blocked, expected_behavior_met: true for all three

Scenarios: schema_drift_missing_transaction_id → required_columns_present fails; invalid_event_type_injection → event_type_domain_check fails; user_id_null_spike → user_id_null_rate fails

Proves: Blocking quality gates catch three categories of bad data and prevent metric computation.

Does NOT prove: That all real-world data quality failures are caught.

Safe wording: "I deliberately injected three failure scenarios — missing column, invalid event type, and null spike — and verified each produces a blocked pipeline run."

outputs/evidence/experiment_assignment_balance.json

Key numbers: assigned_user_count: 20000, treatment: 10094, control: 9906, treatment_share: 0.5047, absolute_imbalance: 0.0047, balance_status: pass

Proves: Deterministic SHA-256 assignment distributes 20,000 users at 50.47% treatment with 0.47 pp imbalance. Balance check passes.

Does NOT prove: That the hash function is perfectly uniform for all possible user IDs.

outputs/evidence/srm_check_report.json

Key numbers: chi_square_stat: 1.7672, p_value: 0.1837, srm_detected: false, status: pass

Proves: Chi-square goodness-of-fit shows no SRM at p<0.001 threshold.

Does NOT prove: Absence of assignment bias for a real deployed experiment.

Safe wording: "SRM check: chi-square 1.77, p=0.18. No sample ratio mismatch detected."

outputs/evidence/cuped_experiment_readout.json

Key numbers:

	Raw	CUPED
Relative lift	427%	420%
P-value	2.43e-44	5.03e-44
Is significant	true	true
Variance reduction	—	0.44%
θ	—	0.00378
Synthetic uplift users	255	255

Proves: CUPED algorithm runs correctly (θ computed via Cov/Var, y_{adj} computed, z-test applied). ExperimentEvaluation row written to DB with all fields.

Does NOT prove: That the lift is real. 255 users were deterministically converted.

Critical caveat: 0.44% variance reduction is real. 420% lift is artifact of synthetic injection.

Safe wording: "The CUPED readout shows 420% relative lift at $p=5e-44$, but that lift is driven by 255 synthetically injected treatment conversions — documented in the artifact. The CUPED variance reduction of 0.44% is honest: pre-event count is a weak covariate for binary conversion."

Unsafe wording: "MetaSignal detected a 420% treatment effect."

outputs/validation/cuped_aa_validation_report.json

Key numbers: runs: 1000, cuped_false_positive_rate: 0.055, mean_variance_reduction_pct: 26.46, ks_statistic: 0.01858, ks_critical_5pct: 0.04301, ks_passed: true, status: pass

Proves: CUPED preserves false-positive control under the null. P-value distribution is Uniform(0,1). Mean variance reduction on synthetic data with $r=0.6$ is 26.5%.

Does NOT prove: That CUPED will achieve 26% variance reduction on any real dataset. That number depends entirely on covariate quality.

Safe wording: "On 1,000 A/A runs with synthetic data where I control covariate correlation at 0.6, CUPED gives 26.5% mean variance reduction and a 5.5% false positive rate — KS test passes."

outputs/validation/cuped_edge_case_validation_report.json

Key numbers: 5/5 cases pass

Case	Applied	VR	Fallback
strong_covariate	yes	50.7%	—
weak_covariate	no	0%	unadjusted
missing_xpre	no	0%	unadjusted
no_variance_xpre	no	0%	unadjusted
imbalanced_arms	yes	44.9%	— (flagged)

Proves: Edge case handling is implemented and all 5 cases behave as designed.

outputs/evidence/guardrail_decision_report.json

Key numbers: primary_metric_lift: 419.7%, p_value: 5.03e-44, guardrails_cleared: false, system_recommendation: HOLD, final_decision: HOLD

Guardrail: observed_relative_increase_pct: 3.1 vs max_allowed: 2.0 → status: breached

Proves: Guardrail-first logic blocks a statistically positive experiment when a guardrail breaches.

Does NOT prove: That the 3.1% refund rate increase is real. It is hardcoded.

Safe wording: "The system recommendation is HOLD despite 420% lift because the refund guardrail simulated a 3.1% increase against a 2.0% tolerance."

outputs/evidence/post_launch_validation_report.json

Key numbers: `right_censoring_flag: true, is_mature: false, maturity_window_days: 30, readout_day: 14, post_launch_status: blocked_before_rollout`

Proves: Right-censoring logic exists. Guardrail read at day 14 is marked immature until day 30.

Does NOT prove: That the refund rate numbers are from real transactions.

`outputs/evidence/anomaly_detection_report.json`

Key numbers: `rows_scanned: 139, threshold: 3.0, dow_adjusted: true, injected_anomaly.detected: true (4x baseline at 2015-07-11), alert_count: 3`

Proves: DOW-adjusted z-score algorithm runs on real metric time series and detects a synthetically injected anomaly.

Does NOT prove: Production precision/recall. Only one injection tested here.

`outputs/evidence/anomaly_backtest_report.json`

Key numbers: `labeled_event_count: 20, precision: 1.0, recall: 0.867, f1: 0.929, dow_suppressed: 3 FPs, synthetic_labeled: true`

Proves: On 20 hand-crafted labeled events, DOW adjustment suppresses 3 FPs while maintaining recall.

Does NOT prove: Production precision/recall. All labels are synthetic.

Safe wording: "On 20 synthetic labeled events, precision is 1.0, recall is 0.87. DOW adjustment suppresses 3 false positives that raw rolling z-score would have fired. These are synthetic labels — I'm not claiming production precision."

`outputs/evidence/causal_trap_detection_report.json`

Key numbers:

- Peeking: p-values at days 3/7/10 = 0.048/0.120/0.034. `peeking_warning: true`
- Novelty: week1=8.4%, week2=1.2%, ratio=7.0. `novelty_flag: true`
- Simpson's: overall=+3.1%, country=IN=-1.2%, platform=ios=-1.8%. `simpsons_flag: true`

Proves: Detection logic implemented. All three flags are stored in ExperimentEvaluation. Surfaced via API.

Does NOT prove: These are from a real experiment. Values are hardcoded in the validation script.

`outputs/evidence/golden_scenario_suite_report.json` **(v0, 8 scenarios)**

Key numbers: `scenario_count: 8, passed_count: 8, status: pass`

Scenarios pass: `metric_registry_exists` (3 rows), `metric_results_computed` (417 rows), `quality_failures_captured` (30 failed checks), `simulation_events_defined` (≥ 5), `assignments_exist` (20,000), `cuped_evaluation_exists`, `guardrail_blocks_positive_primary` (HOLD), `pipeline_runs_logged` (51).

`outputs/evidence/golden_scenario_suite_v1_report.json` **(v1, 12 scenarios)**

Key numbers: `scenario_count: 12, passed_count: 12, status: pass`

Additional v1 scenarios: `clean_ship`, `not_significant_hold`, `cuped_strong/weak/missing` covariate, `delayed` guardrail immature, `delayed` guardrail breach, `novelty/peeking` investigation, `human` override enforcement.

`outputs/evidence/operational_history_60_day_report.json`

Key numbers: day_count: 60, scenario_count: 13, metric_registry_prd_metric_count: 7, conflict_count: 4, backfill_job_count: 4, system_health_report_days: 60, anomaly_alert_count: 2

13 simulation scenarios:

ID	Day	Type	Description
SIM-001	1	pipeline_success	Clean baseline
SIM-002	11	schema_drift	Schema drift blocks compute
SIM-003	12	quality_recovery_backfill	Recovery + backfill after drift
SIM-004	15	null_spike	Null spike blocks compute
SIM-005	20	late_arriving_data	Late data triggers recompute
SIM-006	25	double_counting	Double-counting blocked
SIM-007	28	real_anomaly_ios_outage	Confirmed iOS outage anomaly
SIM-008	35	dow_false_positive_suppressed	DOW suppresses FP
SIM-009	40	metric_definition_change	DAU v2 → v1 deprecated
SIM-010	30	experiment_start	Checkout experiment starts
SIM-011	44	delayed_guardrail_immature	Positive primary, guardrail immature
SIM-012	44	human_override	Reviewer override with reason
SIM-013	50	clean_ship	Clean ship with mature guardrails

Proves: 60-day operational lifecycle with scripted failure scenarios. Support tables created with correct schema.

Does NOT prove: That these are production events. They are scripted.

outputs/validation/override_reason_enforcement_report.json

Key numbers: 2/2 checks pass. invalid_override → HTTP 422, valid_override_with_reason → HTTP 200

Proves: API enforces override_reason at HTTP layer using FastAPI TestClient.

outputs/validation/api_smoke_test_report.json

Key numbers: endpoint_count: 8, passed_count: 8, status: pass

Endpoints tested: /health, /metrics, /experiments, /experiments/EXP-CHECKOUT-FRICTION-001/readout, /data-quality/latest, /evidence/guardrail_decision_report, /validation/cuped_aa_validation_report, /resume-signal

outputs/validation/streaming_prd_v1_validation_report.json

Key numbers: check_count: 43, passed_count: 43, status: pass

Checks: 15 class existence + 14 artifact existence + 13 content checks + 1 parquet/fallback

outputs/streaming/streaming_prd_scenario_report.json

Key numbers: scenario_count: 10, implemented_count: 10, status: pass

All 10 scenarios return implemented: true when the streaming demo runs.

outputs/streaming/stream_batch_reconciliation_report.json

Key fields: streaming_value, batch_authoritative_value (= streaming × 0.94), reconciliation_status: minor_delta, batch_remains_authoritative: true, calibration_note: "Streaming is provisional..."

Does NOT prove: That the 6% delta is from real late events. It is hardcoded as 94% of streaming.

outputs/streaming/replay_run_log.json

Key fields: replay_id: replay_20260401_1100, triggered_by: consumer_recovery_after_two_hour_gap, batch_truth_unchanged: true

Proves: Replay design exists. Batch truth is unchanged by replay.

Does NOT prove: That replay was triggered by a real consumer failure.

outputs/reports/metasignal_prd_completion_report_v1.json

Key fields: status: pass, core_prd_completion_status: implemented_at_repo_v1_evidence_level, streaming_prd_completion_status: implemented_at_repo_v1_evidence_level

claim_boundary (verbatim from artifact): "This remains solo-built, non-production, production-simulated. PRD completion here means implemented as executable repo evidence and artifacts, not real production deployment."

docs/*.md files

Doc	Key content
METASIGNAL_ARCHITECTURE.md	Layer-by-layer architecture. Explicit non-goals list.
METASIGNAL_DEMO_NARRATIVE.md	10-step walkthrough. Artifact reference for each step.
METASIGNAL_INTERVIEW_DEFENSE.md	11 Q&A pairs. Short answers. Superseded by this document.
METASIGNAL_PRD_COMPLETION_MAP.md	Claim boundary. Core + streaming PRD completion evidence list.
METASIGNAL_STREAMING_EXTENSION.md	Streaming positioning. Hard rule: provisional only. 9 capability areas with artifacts.

End of Part 1 (Sections 0–6). Continue in Part 2 (Sections 7–12) for Core Concepts, Demo Walkthroughs, Claim Table, Limitations, Q&A Master Bank, and Cross-Examination.

Section 7 — Core Concepts: Interview Explanation Layer

For every concept: plain language first, then technical depth, then the interview Q+A.

Metric Registry

Plain language: A metric registry is a database of metric definitions where every metric stores not just its name but its formula — specifically the numerator SQL, denominator SQL, denominator type, grain, entity type, and owner. Teams can't use a metric without those fields being declared.

Technical depth: In MetaSignal, `metric_definitions` stores `numerator_sql` and `denominator_sql` as text fields, `denominator_type` as a controlled vocabulary (`distinct_users`, `event_count`, `purchase_events`), `grain` (`daily`, `weekly`), `entity_type` (`user`, `event`), `direction` (`increase_good`, `decrease_good`), `version` (integer), `config_hash` (SHA-256), and `parent_version_id` (self-FK for lineage). `MetricResult` rows store `metric_version` so results are immutably linked to the definition that produced them.

Where implemented: `src/metasignal/db/models.py` → `MetricDefinition`. Alembic migration `61738c5dba04`. Seeded by `seed_core.py`.

Artifact: `outputs/evidence/metric_registry.json`

Interview Q: "Why do you need a metric registry if you can just define metrics in your BI tool?"

Strong answer: "BI tools define metrics in presentation layer — the formula is inside a dashboard or a YAML file. A registry puts the definition in the data layer: versioned, hashed, conflict-detectable. If two teams independently define 'conversion rate' with different denominators, a BI tool won't catch that. The registry forces them to use the same row — and if they need different denominators, they create separate versioned definitions and the conflict detector flags them."

`denominator_sql` and `denominator_type`

Plain language: For a ratio metric like conversion rate, the denominator is what you're dividing by. The `denominator_sql` is the actual SQL expression. The `denominator_type` is a categorical label that tells the system what kind of denominator it is.

Technical depth: MetaSignal defines three denominator types: `distinct_users` (`COUNT DISTINCT user_id`), `event_count` (`COUNT filtered events`), `purchase_events` (`COUNT transactions`). The conflict detector uses `denominator_type` as the categorization field — two metrics in the same family with different `denominator_type` values → conflict flagged.

Why it matters: `purchase_conversion_rate_user` uses `distinct_users` denominator. `purchase_conversion_rate_event` uses `event_count`. They produce different numbers for the same business question. Without explicit `denominator_type`, you can't programmatically detect this conflict.

Interview Q: "Couldn't you just use the same metric name consistently across all dashboards?"

Strong answer: "In theory, yes. In practice, at a real company, metric definitions drift. Team A runs an experiment and uses event-level conversion. Team B runs the same experiment and uses user-level conversion. They get different results and spend two days in meetings. The registry makes the denominator a first-class, queryable field so conflicts can be detected automatically before they cause decision errors."

config_hash

Plain language: A fingerprint of the metric definition. If the SQL changes, the fingerprint changes. This forces a new version rather than a silent in-place update.

Technical depth: Computed as `hashlib.sha256(definition_content.encode()).hexdigest()`. Stored in `MetricDefinition.config_hash`. `MetricResult` rows link to `metric_id + metric_version`. If you modify the numerator SQL and create a new definition row, the new row gets a new hash and a new version, and the `parent_version_id` points to the old row. Historical results still link to the old definition version.

Interview Q: "What happens if someone changes the metric SQL without going through versioning?"

Strong answer: "In MetaSignal v0, there's no enforcement preventing a direct DB update. The `config_hash` is a detection mechanism, not a write guard. A more mature system would add DB-level immutability on the `numerator_sql` column once a definition has results attached to it. That's a noted improvement for v1."

Metric Versioning

Plain language: Each metric has a version number and can have a parent version. When a definition changes, a new version is created. Old results remain linked to the old version.

Technical depth: `MetricDefinition.version` is an integer. `parent_version_id` is a self-FK. `MetricResult.metric_version` stores the version that produced each result. UniqueConstraint on `MetricResult` ensures (`metric_id`, `metric_version`, `grain_period`, `segment_key`) is unique — you can't duplicate results for the same definition+version+period+segment.

Operational history evidence: SIM-009 in the 60-day history shows a metric definition change — "DAU v2 introduced and v1 deprecated." `deprecated_at` is set on the old definition.

Denominator Conflicts

Plain language: Two metrics that measure the same business concept but use different denominators will produce different numbers. The conflict detector finds these automatically.

Technical depth: `detect_metric_conflicts_v0.py` groups `MetricDefinition` rows by `metric_family()` (inferred from name via substring matching). For each family with ≥ 2 active members, if `denominator_type`, `grain`, or `entity_type` differs across members, a conflict is logged. Output: `metric_conflict_report.json` with `'recommended_action': "Require owner review and mark one metric as authoritative."`

Numbers: 1 conflict in core seed state. 4 conflicts in PRD operational history state (more metrics added by seeder).

Data Quality Gates

Plain language: Before you compute a metric, you check that the underlying data is trustworthy. If it's not, you produce no metric — better than a wrong metric.

Technical depth: Quality checks run in `run_data_quality_v0.py`. Check types: `required_columns_present` (checks column list), `user_id_null_rate` (null fraction), `event_type_domain_check` (allowed values). Each check writes a `DataQualityCheck` row with `passed`, `severity`, `expected_value`, `actual_value`, `threshold`, `check_details` (JSONB). Blocking severity → pipeline status=blocked, no `MetricResult` rows written.

Evidence numbers: 5 checks pass on clean data. 3 failure injection scenarios → 3 blocked. 30 failed checks in DB total.

Blocking vs Warning Checks

Plain language: Blocking checks prevent computation entirely. Warning checks allow computation but flag the results.

Technical depth: In MetaSignal v0, all checks are `severity=blocking`. Warning severity is designed for (future reference). A real system would have three tiers: `blocking` (stop pipeline), `warning` (flag results, continue), `info` (log only).

Interview Q: "What would happen if a check was warning instead of blocking?"

Strong answer: "The pipeline would continue, producing `MetricResult` rows with a flag like `quality_passed=false`. Downstream consumers could choose to suppress those results in experiment readouts. The `ExperimentEvaluation` row already has a `guardrail_results` JSONB field where you could record which quality checks were active during the evaluation period."

Failure Injection

Plain language: Deliberately feeding bad data to verify the quality gates catch it.

Technical depth: `run_failure_injection_v0.py` creates three modified copies of the events dataframe in memory, runs the quality check on each, and verifies `status=blocked + expected_behavior_met=true`. The scenarios cover: missing column, invalid event type (domain violation), and null rate spike.

Pipeline Run Logging

Plain language: Every pipeline run (data quality, metric compute, etc.) creates a log row in the DB with status, row counts, and quality check counts.

Technical depth: `PipelineRunLog` stores `job_type`, `run_date`, `started_at`, `completed_at`, `status`, `rows_processed`, `rows_expected`, `quality_checks_passed`, `quality_checks_failed`, `config_hash`, `retry_count`, `triggered_by`. 51 rows exist in live DB. `DataQualityCheck` rows FK to `PipelineRunLog` via `run_id`.

Interview value: When an anomaly fires, you can trace back to the pipeline run that computed the metric and check its quality flags.

Backfills

Plain language: When a quality failure is resolved, affected metric results must be recomputed. A backfill job tracks which dates were recomputed and why.

Technical depth: The `backfill_jobs` support table (created by operational history seeder) stores `trigger_reason`, `affected_dates`, `triggered_by_run_id`, `status`, `rows_recomputed`. 4 backfill rows in operational history state. Triggered by SIM-003 (recovery after schema drift), SIM-005 (late data), and others.

SRM (Sample Ratio Mismatch)

Plain language: If you designed an experiment with a 50/50 split but the actual split is 55/45, something went wrong — the assignment logic, tracking, or data pipeline is broken. Interpreting treatment effects from a broken split is invalid.

Technical depth: MetaSignal uses chi-square goodness-of-fit with 1 degree of freedom. Test statistic: $\chi^2 = (\text{obs}_T - \text{exp}_T)^2 / \text{exp}_T + (\text{obs}_C - \text{exp}_C)^2 / \text{exp}_C$. P-value via `math.erfc(sqrt( $\chi^2/2$ ))`. Threshold: $p < 0.001$. The 0.001 threshold (vs standard 0.05) is used for SRM because SRM is a quality check, not a significance test — you want a lower false positive rate.

Numbers: $\chi^2=1.77$, $p=0.18$, no SRM. The deterministic hash naturally produces near-50/50 splits.

Interview Q: "Why $p < 0.001$ for SRM instead of $p < 0.05$?"

Strong answer: "SRM testing is about catching instrumentation bugs, not testing a scientific hypothesis. You want very high confidence that there's a real problem before you invalidate an experiment. A false SRM alarm at $p < 0.05$ would cause you to discard valid experiments 5% of the time. At $p < 0.001$, you only invalidate when the evidence for a problem is overwhelming."

Deterministic Hash Assignment

Plain language: The same user always gets assigned to the same variant. No random seed. No timestamp. Reproducible forever.

Technical depth: `assign_bucket(experiment_key, user_id)`: SHA-256 of "`{experiment_key}:{user_id}`". Take first 12 hex chars as integer. Divide by 16^{12} . Bucket in $[0, 1)$. If `bucket < treatment_split_pct` → treatment. The `experiment_key` namespace prevents the same user getting the same variant in every experiment — a different `experiment_key` produces a different hash.

Why it matters: A user who sees the treatment today still sees the treatment tomorrow. No variance from re-assignment. No network call needed to look up assignment — it can be recomputed deterministically.

Interview Q: "What if you want to gradually ramp up traffic?"

Strong answer: "You change the `treatment_split_pct` on the experiment row. At 5%, only users with `bucket < 0.05` get treatment. At 50%, users with `bucket < 0.50`. The cutoff changes but the bucket for each user doesn't — so users who saw the treatment at 5% are a subset of those who see it at 50%. That's a valid ramp-up strategy."

CUPED

Plain language: CUPED reduces the noise in your experiment readout by subtracting out the part of the outcome that was predictable from the user's pre-experiment behavior. Less noise → same result with smaller sample size, or more confidence from the same sample size.

Technical depth:

$$\theta = \text{Cov}(Y_{\text{post}}, X_{\text{pre}}) / \text{Var}(X_{\text{pre}}) \quad Y_{\text{CUPED}} = Y_{\text{post}} - \theta \times (X_{\text{pre}} - \text{mean}(X_{\text{pre}}))$$

θ captures how much `Y_post` moves per unit of `X_pre`, on average. Subtracting $\theta \times (X_{\text{pre}} - \text{mean}(X_{\text{pre}}))$ removes that predictable component. The expected value of `Y_CUPED` is the same as `Y_post` (θ adjustment is mean-zero), so the treatment effect estimate is unbiased. The variance of `Y_CUPED` is $\text{Var}(Y_{\text{post}}) \times (1 - \rho^2)$ where ρ is the correlation between `Y_post` and `X_pre`. Variance reduction = ρ^2 .

MetaSignal numbers:

- Covariate: `pre_event_count` (total events per user before experiment start)
- $\theta = 0.00378$ (near zero — covariate is weakly predictive)
- Variance reduction: 0.44% (ρ is near zero)
- A/A validation: with $r=0.6$ synthetic correlation, mean VR = 26.5%

Interview Q: "Why is your CUPED variance reduction only 0.44%?"

Strong answer: "Pre-event count has very low correlation with binary purchase conversion over a 14-day window. CUPED's effectiveness is proportional to ρ^2 — if the covariate doesn't predict the outcome, you get near-zero variance reduction. A better covariate would be the user's purchase conversion rate in the 30 days before the experiment, or their revenue. I chose `pre_event_count` because it's available in the RetailRocket dataset and it demonstrates the algorithm — but it's not the strongest possible covariate. The A/A validation separately demonstrates that with a controlled 0.6 correlation you get 26.5% reduction."

CUPED θ

Plain language: θ is the slope of the linear relationship between pre-period behavior and post-period outcome. It tells you how much to adjust Y_{post} for each unit of X_{pre} .

Technical depth: Computed as sample covariance divided by sample variance of X_{pre} . Pooled across all users (both treatment and control). $\theta = 0.00378$ in the MetaSignal experiment, meaning for every 1 unit increase in pre_event_count , we adjust post-period conversion down by 0.00378. Near-zero $\rightarrow$ adjustment is nearly zero for all users.

A/A Validation

Plain language: Run your analysis on data with no treatment effect (assign users randomly to "treatment" and "control" but don't actually do anything different). If your method is calibrated, you should see $p < 0.05$ about 5% of the time — no more, no less.

Technical depth: `run_cuped_aa_validation_v0.py` uses numpy RNG to generate synthetic pre-period and post-period data with controlled correlation ($r=0.6$ via $Y_{\text{post}} = 0.6 \times X_{\text{pre}} + \text{noise}$). 50/50 random assignment. CUPED applied. 1,000 runs. False positive rate = fraction of runs with CUPED p-value < 0.05 .

Numbers: FPR = 5.5% (target 5%). KS statistic = 0.01858 vs KS critical = 0.04301 $\rightarrow$ passes. Mean VR = 26.5%.

Why KS test: The Kolmogorov-Smirnov test on p-values against Uniform(0,1) checks that the null distribution of p-values is uniform — not concentrated near 0 (inflated FPR) or near 1 (deflated FPR). Passing the KS test is stronger evidence of calibration than just checking the FPR.

CUPED Edge Cases

Five cases, all handled:

Strong covariate ($r \approx 0.75$): Applied, VR=50.7%. The case where CUPED is most useful.

Weak covariate ($r \approx 0.02$): Falls back to unadjusted. $\theta = -0.0028 < \text{threshold } 0.05 \rightarrow$
`cuped_edge_case=weak_covariate, cuped_fallback=unadjusted.`

Missing X_{pre} (>50% users missing pre-period data): Falls back. `cuped_edge_case=missing_xpre.`

Zero variance X_{pre} (all users have identical pre-period value): Falls back. Division by zero avoided.
`cuped_edge_case=no_variance_xpre.`

Imbalanced arms (80/20 split): CUPED still applied, but `cuped_edge_case=imbalanced_arms` flags the unusual split for human review.

Guardrail-First Decisioning

Plain language: A guardrail metric is a harm indicator. If it breaches its tolerance, the experiment should not ship — regardless of how good the primary metric looks. This prevents "grow revenue, ignore refund rate" errors.

Technical depth: `run_guardrail_decision_v0.py` evaluates `guardrail_results` — each guardrail has `status: pass/breached`. `guardrails_cleared = all(g["status"] == "pass")`. If not cleared $\rightarrow$ `system_recommendation = HOLD`. The DecisionLog stores both `system_recommendation` and `final_decision` separately, allowing human override with required written reason.

The critical MetaSignal example: Primary lift = 420%, $p=5e-44$. Refund guardrail breached at 3.1% vs 2.0% tolerance. `final_decision = HOLD`. This is the single strongest behavioral demonstration in the repo.

Interview Q: "Are guardrails just another hypothesis test?"

Strong answer: "No. Guardrails are decision gates, not statistical tests. A hypothesis test asks 'is there a statistically significant effect?' A guardrail asks 'is the effect within an acceptable range?' The threshold is a business tolerance, not a p-value cutoff. You don't need to be confident the guardrail is truly breached — if it's even close to the tolerance, you investigate before shipping. The asymmetry of product harm justifies a more conservative threshold than typical significance testing."

Right-Censoring

Plain language: Some outcome metrics take longer to materialize than the experiment duration. A 14-day experiment can't produce a reliable refund rate if most refunds happen 15-45 days after purchase. The early reading is incomplete — it's right-censored.

Technical depth: `run_post_launch_validation_v0.py` checks `maturity_window_days` against the evaluation day. If `evaluation_day < maturity_window_days` → `right_censoring_flag = true, is_mature = false`. The guardrail result is marked `status: immature_but_breached` — the system knows the breach exists but also knows it may change as more refund data arrives.

Why it matters: A guardrail that appears to pass at day 14 might breach at day 30. Shipping based on the immature reading exposes you to retroactive harm. MetaSignal blocks final decisions until the maturity window closes.

Post-Launch Validation

Plain language: After an experiment decision, verify that the outcome matches expectations. If the system said HOLD, confirm the system ran correctly. If later data arrives, update the assessment.

Technical depth: `post_launch_validation_report.json` stores the final assessment including `post_launch_status: blocked_before_rollout` and the lesson documented in `evidence_statement`.

Anomaly Detection

Plain language: Watch metric time series for unusual spikes or drops that indicate a data pipeline failure, product bug, or real business change.

Technical depth: For each data point at day D: (1) collect up to 8 same-day-of-week historical points from D's history, (2) compute mean and stddev from those, (3) $z = (\text{value} - \text{baseline_mean}) / \text{baseline_std}$, (4) alert if $|z| \geq 3.0$. If <4 same-DOW points available, fall back to 14-day rolling window.

Evidence: 139 time series points scanned. Synthetic anomaly at 2015-07-11 (4x baseline) detected. 3 total alerts.

DOW-Adjusted Z-Score

Plain language: E-commerce metrics are higher on weekdays than weekends, and higher in some weeks than others. Comparing Monday against the overall average fires false alarms on Mondays. Comparing Monday against previous Mondays eliminates that.

Technical depth: The DOW adjustment means the baseline is built from same-weekday historical values (`df["dow"] = df["date"].dt.dayofweek`). Up to 8 same-DOW points are used for mean/stddev. This is not EWMA — it's a simple rolling same-DOW average.

Backtest evidence: 3 `known_dow_false_positive` events — raw z-score would fire ($z=3.2$), DOW-adjusted z-score does not ($z=1.1$). DOW adjustment suppresses all 3.

Anomaly Backtest

Plain language: Test the anomaly detector on labeled examples to measure how well it works.

Technical depth: 20 hand-crafted events across 5 categories. For each event: specify `raw_z` and `dow_z` (what the detector would compute). Fire rule: `dow_z ≥ 3.0`. Compute precision, recall, F1.

Numbers: precision=1.0, recall=0.867, F1=0.929. All labeled events that should fire do fire (except 2 true positives in the ambiguous/below-threshold category). No false positives because DOW adjustment eliminates the `known_dow_fp` events.

Critical caveat: These are synthetic labels. The "expected_fire" for each event is manually set by the script author. This is not production precision/recall.

Peeking

Plain language: If you check your experiment results every day and stop when you first see $p < 0.05$, you will find "significant" results much more than 5% of the time, even under the null. This is called peeking.

Technical depth: The `peeking_warning` in MetaSignal fires when the significance conclusion changes across interim reads. Three simulated reads: day 3 ($p=0.048$, `sig=true`), day 7 ($p=0.12$, `sig=false`), day 10 ($p=0.034$, `sig=true`). Conclusion changed → `peeking_warning=true`.

What peeking_warning doesn't solve: It documents that peeking occurred. It doesn't apply sequential testing corrections like always-valid p-values or mSPRT. That would be the next implementation step.

Interview Q: "How do you fix peeking?"

Strong answer: "Three approaches: (1) pre-register a fixed sample size and stick to it — don't look until you hit it; (2) use sequential testing methods like always-valid confidence intervals or SPRT that remain valid under repeated looks; (3) apply Bonferroni/Holm correction to the interim readings. MetaSignal flags peeking but doesn't apply correction — that would be the next implementation step."

Novelty Effect

Plain language: Users are curious about new features and engage more than usual right after launch. Week-1 lift looks amazing. Week-2 lift is much lower because the novelty wore off.

Technical depth: `novelty_flag = week_1_lift / week_2_lift > 2.0`. In MetaSignal: $8.4\% / 1.2\% = 7.0 > 2.0$ → `novelty_flag=true`. Stored in ExperimentEvaluation as a Boolean column. Surfaced via API.

What to do when flagged: Extend the experiment duration to see if the lift stabilizes. Don't ship on the basis of week-1 lift alone.

Simpson's Paradox

Plain language: An overall positive result can hide a negative result in important subgroups. Shipping based on the overall result would harm those subgroups.

Technical depth: `simpsons_flag = overall_lift > 0 AND any(segment_lift < 0 for segment != "overall")`. MetaSignal shows: overall=+3.1%, country=IN=-1.2%, country=US=+4.4%, platform=ios=-1.8%, platform=web=+3.9%. Flag fires because IN and iOS are negative while overall is positive.

Why it matters: If your user base is 80% India and the treatment harms Indian users, the overall positive result is misleading. Shipping harms your primary market.

Decision Audit Logs

Plain language: Every decision is recorded: what the system recommended, what the human decided, who reviewed it, and why if they overrode.

Technical depth: `DecisionLog` has `system_recommendation`, `final_decision`, `reviewer_name`, `override_reason` (nullable), `decision_notes`. The API enforces `override_reason` is non-null when `final_decision != system_recommendation` (HTTP 422 otherwise). Two decision rows in live DB, both HOLD, both no override.

Override Enforcement

Plain language: If a human wants to override the system's recommendation, they must explain why in writing. The API refuses the request without a reason.

Technical depth: `POST /decisions/validate-override`. Logic: `is_override = human_decision in {"override_ship", "override_hold"} OR (system_recommendation != human_decision)`. If `is_override AND NOT override_reason` → HTTP 422. Verified by `test_override_enforcement_v1.py` via FastAPI `TestClient`.

Streaming Provisionality

Plain language: Streaming results are early warnings, not final decisions. They can be wrong because they're incomplete — late events haven't arrived, duplicates haven't been deduplicated, batch hasn't confirmed the numbers.

Technical depth: From `METASIGNAL_STREAMING_EXTENSION.md`: "Streaming alerts may trigger investigation or pause-for-review, but never final experiment decisions." All streaming metric snapshots have `provisional_label=true`. Validation check `all_snapshots_provisional` verifies this. Stream-batch reconciliation tracks the delta between provisional streaming value and batch authoritative value.

DLQ (Dead Letter Queue)

Plain language: Events that fail validation are quarantined in a DLQ instead of being silently dropped or silently included. This preserves them for debugging and root cause analysis.

Technical depth: `DeadLetterQueueWriter` stores events with `error_code` (`MISSING_REQUIRED_FIELD`, `INVALID_VARIANT`, `SCHEMA_TYPE_MISMATCH`) and `error_detail`. Writes `dlq_events.json` (always) and `dlq_events.parquet` (if pandas + pyarrow available). DLQ count is tracked in `StreamValidator` and surfaced in streaming health report.

Streaming scenarios that produce DLQ events: Window 4 (malformed events — 10 each of missing `experiment_id`, invalid variant, poison type mismatch). DLQ count > 0 verified in `streaming_prd_v1_validation`.

Consumer Lag

Plain language: How far behind is the event processing pipeline? If events are arriving at 9:00 AM but being processed at 9:20 AM, you have 20-minute consumer lag. High lag means provisional metrics are stale and provisional anomaly alerts may be false.

Technical depth: `ConsumerLagMonitor` computes `processing_time - event_time` for each event per window. Reports P50, P95, P99, max per window. Alert level = "investigate" if P95 > 300 seconds (5 minutes). Window 7: injected lag of 1200-1500 seconds → fires investigate.

Why lag suppresses anomaly alerts: `ProvisionalAnomalyDetector` checks `lag_suppressed = lag["p95_lag_seconds"] > 300`. If suppressed, the `batch_confirmation_status` for the provisional alert is set to "suppressed" — the system knows the anomaly may be an artifact of processing delay, not a real metric movement.

Late Events

Plain language: An event that arrives later than expected — maybe a mobile app was offline and batched up events. If you compute metrics based on events received up to time T, late events arriving after T will be missed.

Technical depth: `StreamValidator` marks an event as late if `event_time < window_start - timedelta(minutes=15)`. Late events are excluded from valid event count and go to a separate `late_events` list. The batch authoritative layer handles late events in its computation (it waits for the full day's events). `late_event_report.json` stores the count and handling note: "batch_authoritative_layer_handles_late_events."

Why it matters: If you naively use streaming counts for experiment decisions, late events cause you to undercount conversions in the streaming layer. The batch layer accounts for late arrivals but the streaming layer can't.

Stream Quality Checks

Plain language: Per-window quality checks on the streaming data itself — minimum event count, malformed rate, duplicate rate, late event rate, consumer lag.

Technical depth: `StreamQualityChecker.run()` produces per-window checks: `minimum_valid_events > 100` (pass/fail), `malformed_rate <= 0.01` (pass/warn), `duplicate_rate <= 0.002` (pass/warn), `late_event_rate <= 0.002` (pass/warn), `consumer_lag_p95 <= 300` (pass/warn). 60 checks total (5 checks × 12 windows). Verified by streaming validation: `stream_quality_checks_present` check requires ≥ 50 .

Stream-Batch Reconciliation

Plain language: After a streaming window closes, compare the streaming metric value to what the batch layer computes for the same period. The difference tells you how much to distrust the provisional streaming number.

Technical depth: `StreamBatchReconciler` computes `streaming_value` from valid events, then `batch_value = streaming_value × 0.94` (simulated 6% delta). `absolute_delta = |streaming - batch|`. `relative_delta = absolute_delta / batch`. If `relative_delta < 0.15` → `reconciliation_status = minor_delta`. `batch_remains_authoritative=true` always.

Calibration note (from artifact): "Streaming is provisional count ratio; batch is authoritative and applies grain, late-event inclusion, and CUPED."

Replay

Plain language: If the event consumer crashed and you missed two hours of events, you need to reprocess those events to update your streaming artifacts.

Technical depth: `ReplayManager.replay()` returns a structured replay log: `replay_id`, `triggered_by`, `window_range_replayed`, `events_reprocessed`, `artifacts_updated`, `batch_truth_unchanged=true`. This is simulated — the actual replay logic isn't fully implemented, but the design exists.

Section 8 — Full Demo Walkthrough

2-Minute Demo

What to say:

"Let me show you MetaSignal in two minutes. The run command is `PYTHONPATH=. python3 scripts/run_full_demo_v0.py`. That runs 24 scripts in sequence.

>

*The most important thing to look at afterward is `outputs/evidence/guardrail_decision_report.json`. Open it. You'll see: primary metric lift is 420%, $p=5e-44$, statistically significant. System recommendation: **HOLD**. Why? Because the refund guardrail breached — 3.1% observed versus 2.0% tolerance. Guardrail-first means that even with 420% lift, the system says **HOLD**.*

>

Second artifact to check: `outputs/validation/cuped_aa_validation_report.json`. CUPED false positive rate is 5.5%, KS test passes, mean variance reduction 26.5% on synthetic data. That tells you the statistical method is calibrated.

>

Third: `outputs/streaming/streaming_prd_scenario_report.json`. 10 streaming scenarios, 10 implemented. That's the streaming layer.

>

Everything traces to a DB row or a JSON file."

Key artifact to show: `outputs/evidence/guardrail_decision_report.json`

5-Minute Demo

Step 1 — Setup (30s)

```
# Make sure DB is running and events parquet exists # Then: PYTHONPATH=. python3
scripts/run_full_demo_v0.py
```

Expected output: 24 numbered commands, each printing completion message. Final line: `=== MetaSignal Full Demo v0 completed successfully ===`

Step 2 — Data substrate (30s)

Open `outputs/evidence/data_quality_service_report.json`.

```
"rows_processed": 2756101, "checks_passed": 5, "checks_failed": 0, "status": "success"
```

Say: "2.75 million RetailRocket events. 5 quality checks. All pass. Pipeline run logged to DB."

Step 3 — Metric governance (45s)

Open `outputs/evidence/metric_registry.json`. Show the two conversion rate metrics with different denominator types.

Open `outputs/evidence/metric_conflict_report.json`.

Say: "Two active metrics with the same business meaning but different denominators. Conflict detected automatically."

Step 4 — Assignment + SRM (30s)

Open `outputs/evidence/experiment_assignment_balance.json` then `srm_check_report.json`.

Say: "20,000 deterministic SHA-256 assignments. 50.47% treatment. SRM check: chi-square 1.77, $p=0.18$. No sample ratio mismatch."

Step 5 — CUPED evaluation (60s)

Open `outputs/evidence/cuped_experiment_readout.json`.

Say: "CUPED readout: 420% relative lift, $p=5e-44$. Variance reduction 0.44% — the covariate is weak. 255 users deterministically injected as treatment conversions for the portfolio demo — documented in the artifact. Check `outputs/validation/cuped_aa_validation_report.json` for the 1,000 A/A runs that validate the algorithm."

Step 6 — Guardrail + HOLD (45s)

Open `outputs/evidence/guardrail_decision_report.json`.

Say: "Positive primary metric. Refund guardrail: 3.1% vs 2.0% tolerance. HOLD. This is guardrail-first decisioning — the guardrail result is dispositive. Check `outputs/evidence/post_launch_validation_report.json`: the guardrail is also right-censored at day 14, maturity window is 30 days."

Step 7 — Streaming (30s)

```
PYTHONPATH=. python3 scripts/show_streaming_demo.py
```

Then open `outputs/streaming/streaming_prd_scenario_report.json`: 10/10 scenarios pass.

10-Minute Deep Demo

Phase 1: Data and governance (3 min)

```
PYTHONPATH=. python3 scripts/run_data_quality_v0.py # → outputs/evidence/data_quality_service_report.json
(5/5 pass) PYTHONPATH=. python3 scripts/run_failure_injection_v0.py # →
outputs/evidence/failure_injection_report.json (3/3 blocked)
```

Walk through `failure_injection_report.json`: `show schema_drift → required_columns_present` fails, `invalid_event_type → domain check` fails, `null_spike → user_id_null_rate` fails. All `expected_behavior_met: true`.

Then:

```
PYTHONPATH=. python3 scripts/compute_metrics_v0.py PYTHONPATH=. python3
scripts/detect_metric_conflicts_v0.py # → outputs/evidence/metric_conflict_report.json
```

Show the conflict: two active conversion rate metrics with denominator_type `event_count` vs `distinct_users`.

Phase 2: Assignment and SRM (1.5 min)

```
PYTHONPATH=. python3 scripts/generate_assignments_v0.py # →
outputs/evidence/experiment_assignment_balance.json (20,000 users, pass) PYTHONPATH=. python3
scripts/run_srm_check_v0.py # → outputs/evidence/srm_check_report.json (chi-sq=1.77, p=0.18, pass)
```

Explain deterministic hash: "Same user, same variant, forever. No database lookup needed."

Phase 3: CUPED + A/A validation (2 min)

```
PYTHONPATH=. python3 scripts/run_cuped_v0.py # → outputs/evidence/cuped_experiment_readout.json
PYTHONPATH=. python3 scripts/run_cuped_aa_validation_v0.py # →
outputs/validation/cuped_aa_validation_report.json
```

Walk through `cuped_readout`: explain synthetic uplift (255 users), $\theta=0.00378$, $VR=0.44\%$, raw vs CUPED comparison.

Walk through AA validation: $FPR=5.5\%$, KS passes, mean $VR=26.5\%$.

Then run edge case suite:

```
PYTHONPATH=. python3 scripts/run_prd_core_validation_suite_v1.py # →
outputs/validation/cuped_edge_case_validation_report.json (5/5 pass)
```

Phase 4: Guardrail and right-censoring (1.5 min)

```
PYTHONPATH=. python3 scripts/run_guardrail_decision_v0.py # →
outputs/evidence/guardrail_decision_report.json (HOLD) PYTHONPATH=. python3
```

```
scripts/run_post_launch_validation_v0.py # → outputs/evidence/post_launch_validation_report.json
(immature_but_breached)
```

Walk through guardrail_decision_report.json: highlight primary_positive=true AND guardrails_cleared=false AND system_recommendation=HOLD.

Phase 5: Anomaly + causal traps (1 min)

```
PYTHONPATH=. python3 scripts/run_anomaly_detector_v0.py # → outputs/evidence/anomaly_detection_report.json
(injected detected)
```

Show causal trap report from prd_core_validation_suite: peeking=true, novelty=true, simpsons=true.

Phase 6: Streaming (1 min)

```
PYTHONPATH=. python3 scripts/show_streaming_demo.py PYTHONPATH=. python3
scripts/validate_streaming_prd_v1.py # → outputs/validation/streaming_prd_v1_validation_report.json (43/43
pass)
```

Walk through streaming_prd_scenario_report.json: 10/10 implemented.

Show stream_batch_reconciliation_report.json: streaming provisional vs batch authoritative, batch_remains_authoritative=true.

Commands reference

Command	Expected output	Key artifact
PYTHONPATH=. python3 scripts/run_full_demo_v0.py	24 scripts, all exit 0	All outputs/
PYTHONPATH=. python3 scripts/run_metasignal_prd_complete_v1.py	status: pass	metasignal_prd_completion_report_v1.json
PYTHONPATH=. python3 scripts/show_streaming_demo.py	10 streaming metrics printed	All outputs/streaming/
PYTHONPATH=. python3 scripts/validate_streaming_prd_v1.py	43/43 pass	streaming_prd_v1_validation_report.json
PYTHONPATH=. python3 scripts/smoke_test_api_v0.py	8/8 pass	api_smoke_test_report.json
PYTHONPATH=. python3 scripts/run_prd_core_validation_suite_v1.py	4 reports written	cuped_edge_case, golden_v1, causal_traps, anomaly_backtest

Section 9 — Implemented vs Simulated vs Partially Verified vs Future

Complete Claim Table

Area	Claim	Status	Evidence Path	Safe Wording	Unsafe Wording	Notes
Data	RetailRocket 2.75M events	IMPLEMENTED	data_quality_service_report.json	"Real public dataset, 2.75M rows"	"Production traffic"	Public e-commerce dataset
Data	Historical dates ~2015	IMPLEMENTED	cuped_experiment_readout.json (period dates)	"RetailRocket dataset from 2015"	"Current experiment data"	
DB	PostgreSQL + SQLAlchemy ORM	IMPLEMENTED	models.py, alembic migrations	"Real PostgreSQL DB with ORM"	"Enterprise DB"	Local dev DB
DB	9 Alembic-migrated tables	IMPLEMENTED	Both migration files	"9 tables in 2 migrations"	"Complete schema"	Support tables not in Alembic
DB	4 support tables (raw SQL)	PARTIALLY VERIFIED	seed_operational_history_v1.py	"4 tables created by seeder, not Alembic"	"Fully migrated schema"	Raw SQL, recreated each seed run
Metrics	3 core metric definitions	IMPLEMENTED	metric_registry.json	"3 active metrics in registry"	"7 metrics" without context	7 is post-operational-history count
Metrics	7 PRD-level metrics	SIMULATED	operational_history_60_day_report.json	"7 metrics in PRD scenario state"	"7 production metrics"	Added by seeder
Metrics	Denominator conflict detection	IMPLEMENTED	metric_conflict_report.json	"Conflict detector flags active metrics with mismatched denominators"	"We detect all metric conflicts"	
Metrics	Config hash + versioning	IMPLEMENTED	models.py, metric_registry.json	"SHA-256 hash per definition, self-FK version lineage"	"Immutable definitions"	No DB-level write guard
Quality	5 data quality checks	IMPLEMENTED	data_quality_service_report.json	"5 checks on 2.75M rows, all pass"	"Comprehensive DQ coverage"	Only 3 check types implemented
Quality	3 failure injection scenarios	SIMULATED	failure_injection_report.json	"3 bad-data scenarios → 3 blocked runs"	"All failure modes covered"	

Area	Claim	Status	Evidence Path	Safe Wording	Unsafe Wording	Notes
Experiment	Deterministic SHA-256 assignment	IMPLEMENTED	generate_assignments_v0.py, experiment_assignment_balance.json	"SHA-256 hash, same user = same variant"	"Distributed assignment system"	Local implementation only
Experiment	20,000 user assignments	IMPLEMENTED	experiment_assignment_balance.json	"20,000 users assigned, 50.47% treatment"	"20,000 real users"	RetailRocket users, not real experiment participants
Experiment	SRM chi-square test	IMPLEMENTED	srm_check_report.json	"Chi-sq=1.77, p=0.18, no SRM"	"SRM check in production"	
CUPED	CUPED algorithm implementation	IMPLEMENTED	run_cuped_v0.py, cuped_experiment_readout.json	"Cov/Var theta, z-test, ExperimentEvaluation row written"	"CUPED in production"	
CUPED	255 synthetic uplift users	SIMULATED	cuped_experiment_readout.json (synthetic_uplift_users)	"255 users deterministically converted to simulate effect"	Never hide this	Must disclose
CUPED	0.44% variance reduction	IMPLEMENTED	cuped_experiment_readout.json	"0.44% VR — weak covariate"	"26% variance reduction in experiment"	26% is from A/A validation only
CUPED	420% relative lift	SIMULATED	cuped_experiment_readout.json	"420% lift, driven by synthetic injection"	"Treatment effect is 420%"	Effect is not real
CUPED	A/A validation 1,000 runs	IMPLEMENTED	cuped_aa_validation_report.json	"1,000 A/A runs, FPR=5.5%, KS passes"	"Validated on real data"	Synthetic numpy data
CUPED	5 edge cases	IMPLEMENTED	cuped_edge_case_validation_report.json	"5 edge cases, all pass"		
Guardrails	Guardrail-first decision logic	IMPLEMENTED	run_guardrail_decision_v0.py, guardrail_decision_report.json	"HOLD despite positive primary due to guardrail breach"		

Area	Claim	Status	Evidence Path	Safe Wording	Unsafe Wording	Notes
Guardrails	3.1% refund rate breach	SIMULATED	guardrail_decision_report.json	"3.1% simulated breach against 2.0% tolerance"	"Guardrail fired on real refund data"	No return_initiated in RetailRocket
Guardrails	Right-censoring	IMPLEMENTED	post_launch_validation_report.json	"Guardrail marked immature at day 14, maturity window 30 days"		
Guardrails	Override reason API enforcement	IMPLEMENTED	override_reason_enforcement_report.json	"API returns 422 without override_reason"		
Anomaly	DOW-adjusted z-score	IMPLEMENTED	run_anomaly_detector_v0.py, anomaly_detection_report.json	"Same-weekday rolling baseline, threshold $z \geq 3$ "		
Anomaly	Synthetic injection detected	SIMULATED	anomaly_detection_report.json	"4x baseline injection at day 45 detected"	"Production anomaly detection"	
Anomaly	Backtest precision=1.0 recall=0.87	SIMULATED	anomaly_backtest_report.json	"On 20 synthetic labeled events"	"Production precision/recall"	All labels are hand-crafted
Causal	Peeking warning	SIMULATED	causal_trap_detection_report.json	"Peeking detected via simulated interim reads"	"Real peeking caught in experiment"	Hardcoded p-values
Causal	Novelty flag	SIMULATED	causal_trap_detection_report.json	"Novelty flag: week1/week2 ratio=7"		Hardcoded
Causal	Simpson's paradox flag	SIMULATED	causal_trap_detection_report.json	"Simpson's: overall +3.1%, IN=-1.2%, iOS=-1.8%"		Hardcoded
Streaming	asyncio simulation	IMPLEMENTED (SIMULATED events)	streaming_prd_v1.py	"asyncio-based event simulation, 12 windows x 500 events"	"Kafka streaming"	No Kafka

Area	Claim	Status	Evidence Path	Safe Wording	Unsafe Wording	Notes
Streaming	15 streaming classes	IMPLEMENTED	streaming_prd_v1.py, streaming_validation	"15 classes exist and are tested"		
Streaming	10 failure scenarios	SIMULATED	streaming_prd_scenario_report.json	"10 scenarios injected and all produce expected behavior"	"10 production failure modes handled"	
Streaming	43/43 validation checks	IMPLEMENTED	streaming_prd_v1_validation_report.json	"43 checks: class existence, artifact existence, content"		
Streaming	DLQ JSON + parquet	IMPLEMENTED / PARTIALLY VERIFIED	dlq_events.json, dlq_events.parquet	"DLQ JSON always written. Parquet written if pyarrow available."	"DLQ in production"	
Streaming	Provisional-only snapshots	IMPLEMENTED	streaming_metric_snapshots.json	"All snapshots labeled provisional_label=true, never final decisions"		
Streaming	Stream-batch reconciliation	SIMULATED	stream_batch_reconciliation_report.json	"Batch delta simulated at 6%, batch remains authoritative"	"Real reconciliation"	0.94 factor hardcoded
Streaming	Replay	SIMULATED	replay_run_log.json	"Replay log structure exists, batch truth unchanged"	"Full replay implemented"	Log only, not full replay
API	FastAPI 10 endpoints	IMPLEMENTED	app.py, api_smoke_test_report.json	"10 endpoints, 8/8 smoke test pass"		
API	Path injection protection	IMPLEMENTED	app.py	"Artifact names sanitized (., / removed)"		

Area	Claim	Status	Evidence Path	Safe Wording	Unsafe Wording	Notes
CI	GitHub Actions	IMPLEMENTED	.github/workflows/metasignal-validation.yml	"CI runs on push to main, Python 3.11, lightweight deps"	"Full CI/CD"	No DB in CI
CI	DB in CI	FUTURE / NOT IMPLEMENTED	—	"CI does not spin up PostgreSQL"	"Full DB tested in CI"	
Ops	60-day operational history	SIMULATED	operational_history_60_day_report.json	"60 scripted days, 13 scenarios"	"60 days of production history"	
Ops	13 simulation scenarios	SIMULATED	operational_history_60_day_report.json	"13 labeled scenarios, SIM-001..SIM-013"		
Infra	Kafka	FUTURE / NOT IMPLEMENTED	—	"No Kafka. asyncio simulation only."	Never claim Kafka	
Infra	Real production deployment	FUTURE / NOT IMPLEMENTED	—	"Local development only"	"Deployed"	
Infra	Frontend / dashboard	FUTURE / NOT IMPLEMENTED	—	"No frontend. API and JSON artifacts only."		
Infra	Real users / analysts	FUTURE / NOT IMPLEMENTED	—	"No real users. Solo built."	"Real analysts used this"	
Infra	Spark / Flink	FUTURE / NOT IMPLEMENTED	—	"No distributed compute"		

Section 10 — Limitations and Honest Gaps

How to use this section

For each limitation, this section gives: what it is, why it matters, how to defend it honestly, what NOT to say, and what improvement looks like.

L1: Synthetic treatment effect (255 injected uplift users)

What it is: The RetailRocket dataset has no real A/B experiment. To produce a non-null CUPED readout, 255 treatment users are deterministically converted via `deterministic_uplift()`.

Why it matters: The 420% lift is an artifact of injection, not a real behavioral difference. An interviewer who reads the code will know this.

How to defend it: "The treatment effect is intentionally synthetic and fully documented — in the code, in the artifact, and in the CUPED readout JSON under `synthetic_uplift_users`. A real experiment would use actual treatment/control data. I used injection to produce a non-trivial readout for portfolio purposes. The statistical machinery (θ computation, z-test, CI computation, evaluation row persistence) is real."

What NOT to say: "The experiment shows a 420% treatment effect." This misleads people into thinking the effect is real.

Improvement: Use a dataset with a real embedded A/B experiment (e.g., Booking.com hotel ranking test dataset, or a synthetic dataset where the DGP is explicitly stated).

L2: CUPED variance reduction is 0.44%

What it is: The pre-event count covariate has near-zero correlation with binary purchase conversion over 14 days. $\theta=0.00378$. CUPED does almost nothing here.

Why it matters: If someone asks "how much does CUPED improve your experiment?", citing the 26% figure from A/A validation is misleading. The honest answer requires distinguishing the actual experiment from the validation.

How to defend it: "The 0.44% is honest. CUPED's effectiveness depends entirely on covariate quality. Pre-event count doesn't strongly predict binary conversion over a 14-day window — it's a weak covariate. The A/A validation at 26% uses synthetic data with controlled $r=0.6$ correlation, which is a best-case scenario. In a real implementation, I'd use the user's prior conversion rate or prior revenue as the covariate — much stronger predictors."

What NOT to say: "CUPED reduces variance by 26% in this experiment." That number is from the A/A simulation only.

L3: Refund guardrail breach is hardcoded

What it is: The 3.1% refund rate increase is a hardcoded value in `run_guardrail_decision_v0.py`. RetailRocket doesn't contain `return_initiated` events in meaningful volume.

How to defend it: "The refund data isn't in RetailRocket so I simulated a guardrail breach to demonstrate the guardrail-first logic. The behavior being demonstrated — HOLD when guardrail breaches, regardless of primary metric — is real. The specific 3.1% number is the simulation."

L4: Streaming is asyncio simulation, not Kafka

What it is: `AsyncSimulatorEventSource` generates events via a Python async generator. No message broker, no network, no real consumer group.

Why it matters: The streaming architecture demonstrates design thinking but not production-grade streaming capability. The `EventSource` interface is swappable, but the swap hasn't been made.

How to defend it: "I chose asyncio simulation to demonstrate the architecture and failure scenarios without requiring Kafka infrastructure. The `EventSource` abstract class is designed to be swappable — the consumer depends on the interface. If I were to extend this, I'd implement `KafkaEventSource(EventSource)` consuming from a real Kafka topic, and the rest of the pipeline would work unchanged."

L5: CI doesn't spin up a full DB

What it is: The GitHub Actions workflow installs lightweight Python deps (numpy, fastapi, httpx, pandas, pyarrow, sqlalchemy, pg8000) but doesn't configure a PostgreSQL instance. Scripts requiring live DB aren't in CI.

Why it matters: The scripts that require the DB (CUPED, SRM, guardrail decision) are not CI-gated. A regression in DB-dependent code wouldn't be caught by CI.

How to defend it: "CI validates the statistical and streaming logic that doesn't require a live DB — the core validation suite and streaming validation. A production-grade CI would add a PostgreSQL service container and run the full pipeline. That's a natural next step."

L6: Anomaly backtest uses 20 synthetic labeled events

What it is: The 20 events in `run_anomaly_backtest_v1()` have hand-crafted z-scores and labels. The precision=1.0, recall=0.87 numbers are not from production data.

How to defend it: "The backtest proves that the DOW adjustment algorithm behaves correctly on a set of known cases — it fires when it should and doesn't fire when it shouldn't. The labels are synthetic by design. The artifact explicitly states `synthetic_labeled: true` and `claim_boundary: 'Precision/recall measured on synthetic injected anomaly events only.'`"

L7: Support tables not in Alembic

What it is: `metric_conflicts`, `backfill_jobs`, `anomaly_alerts`, `system_health_reports` are created by raw SQL in `seed_operational_history_v1.py`.

How to defend it: "These tables exist for operational history simulation and are recreated on each seed run. They're not part of the core experiment lifecycle schema. If they were promoted to production use, they'd be added to Alembic migrations. The 9 core tables that matter — metric definitions, results, experiments, assignments, evaluations, decisions — are all Alembic-managed."

L8: Override reason not enforced at DB level

What it is: The DB doesn't have a constraint requiring `override_reason` when `final_decision != system_recommendation`. The enforcement is at the API layer only.

How to defend it: "API-level enforcement is the right boundary for business logic constraints that depend on conditional logic (`override_reason` required only when decisions differ). A pure DB constraint can't express that conditionality. In a real system, you might add an application-level trigger in addition to the API check."

L9: No frontend

What it is: No dashboard, no UI. Output is JSON artifacts and a CLI.

How to defend it: "The system demonstrates platform-level thinking — the API layer and JSON artifacts are the interfaces that would feed a dashboard. Building the dashboard was out of scope — the goal was to show the backend logic. `FastAPI + /resume-signal` endpoint is the starting point for a frontend integration."

L10: PRD completion is "repo evidence level"

What it is: From `metasignal_prd_completion_report_v1.json`: `core_prd_completion_status: "implemented_at_repo_v1_evidence_level"`. This is not enterprise production completion.

How to defend it: Use the exact phrase from the artifact: "implemented at repo evidence level — not production deployment." The PRD completion shows that every specified behavior produces a verifiable artifact. That's meaningful for a solo portfolio project.

Section 11 — Interview Q&A Master Bank

Category 1: Overview / Project Framing

Q1: What is MetaSignal in one sentence?

MetaSignal is a solo-built, production-simulated experimentation intelligence platform with metric governance, CUPED, guardrail-first decisioning, and a streaming early-warning layer — no real users, every claim traceable to a JSON artifact or DB row.

Q2: Why is it called "production-simulated"?

Because it handles realistic production failure modes and produces machine-readable evidence — but it runs against a public dataset with no real users or live deployment. The distinction matters. I don't want to overclaim.

Q3: What was the most difficult part to build?

Getting CUPED right with honest validation. The algorithm is simple; knowing whether it's calibrated requires a separate A/A suite. The 1,000-run A/A with KS test was the part that required the most statistical discipline.

Q4: If this is just a portfolio project, why does it matter?

Because most data science portfolios show models. This shows infrastructure — the scaffolding that makes models trustworthy and decisions auditable. Companies that run A/B tests at scale don't struggle to compute p-values; they struggle with metric governance, assignment validity, guardrail logic, and anomaly detection. This project demonstrates all of those.

Q5: How long did this take to build?

Solo project. Built iteratively from core schema outward — DB → metrics → experiment → CUPED → guardrails → anomaly → streaming.

Q6: Is this deployed anywhere?

No. Local PostgreSQL, local Python. The API runs locally. Not deployed to any cloud.

Q7: Can I run this myself?

Yes. You need PostgreSQL and Python. The run command is `PYTHONPATH=. python3 scripts/run_full_demo_v0.py`. The complete pipeline runs in one command.

Category 2: Architecture

Q8: Walk me through the architecture.

Nine layers. Event substrate (RetailRocket parquet, 2.75M rows) → DB + migrations → core seeding (3 metrics, 1 experiment) → data quality gate → metric computation → experiment assignment → SRM check → CUPED evaluation → guardrail decisioning → anomaly detection → API. Plus a separate streaming layer that runs as a provisional overlay on the batch layer.

Q9: Why batch-authoritative?

Because streaming values are provisional — late events haven't arrived, duplicates haven't been fully deduplicated, and the streaming computation doesn't apply CUPED or grain normalization. The batch layer applies all of those. Streaming

gives speed for early warning. Batch gives authority for final decisions.

Q10: Why not use a microservices architecture?

For a portfolio project, a monolith is appropriate. The architecture is designed to be decomposable — each layer is a separate script with a defined interface (reads DB, writes DB + artifact). In production, each layer would map to a separate service. The interfaces are already clear.

Q11: How does the API fit in?

The API exposes DB state and JSON artifacts via HTTP endpoints. It's not the computation layer — computation happens in scripts. The API is the consumption layer — for a dashboard, for a CI test, or for an interviewer verifying the outputs.

Category 3: Data and RetailRocket

Q12: What is RetailRocket?

A real public e-commerce dataset. User interactions (views, add-to-cart, transactions) for a real e-commerce company, anonymized. About 2.75 million events. Available on Kaggle. Historical dates ~2015.

Q13: Why RetailRocket and not synthetic data?

Because a real dataset has realistic distributions — skewed user activity, natural seasonality, realistic conversion rates (~0.65% for the user-level metric). A fully synthetic dataset would look artificially clean. RetailRocket gives realistic behavior without the realism of a production data feed.

Q14: What are the event types?

`view`, `addtocart`, `transaction`. That's it. No `return_initiated` — which is why the refund guardrail breach has to be simulated.

Q15: What does standardization do to the RetailRocket data?

Normalizes column names to `event_timestamp`, `event_date`, `user_id`, `event_type`, `item_id`, `transaction_id`. Saves to parquet for efficient columnar reads.

Q16: Why are experiment dates in 2015?

Because the RetailRocket dataset is from 2015. I use the actual event dates from the dataset for the experiment window (2015-05-10 to 2015-05-24). This is not a bug — it's the honest representation of when those events occurred.

Category 4: Metric Registry / Denominator Governance

Q17: What's the problem with denominator inconsistency?

The same business question — "what is our conversion rate?" — answers differently depending on the denominator. `transactions / views` gives an event-level ratio. `converting_users / total_users` gives a user-level ratio. They can differ by 10-50x for the same data. If teams use different denominators to evaluate the same experiment, they get different results and may reach different conclusions.

Q18: How does MetaSignal prevent this?

By requiring `denominator_type` as a controlled field in the metric definition, and by running a conflict detector that flags families where active metrics use different denominator types.

Q19: What is `config_hash` and why does it matter?

SHA-256 of the metric definition content. If the SQL changes, the hash changes, forcing a new version. MetricResult rows store `metric_version`, linking each result to the exact definition version that produced it. This prevents retroactive definition changes from corrupting historical results.

Q20: Can a metric be deprecated?

Yes. `is_active=false`, `deprecated_at` timestamp. SIM-009 in the 60-day history shows DAU v1 being deprecated when DAU v2 is introduced. Both remain queryable — the deprecated version's historical results are still valid.

Q21: What's the difference between 3 metrics and 7 metrics?

The core seed (`seed_core.py`) creates 3 metrics. The operational history seeder (`seed_operational_history_v1.py`) adds 4 more PRD-level metrics for the 60-day simulation scenarios. The 7-metric state exists only in the PRD completion / operational history run, not in the basic v0 demo.

Category 5: Data Quality / Failure Injection

Q22: What happens when a quality check fails?

A `DataQualityCheck` row is written with `passed=false`. If `severity=blocking`, the pipeline log records `status=blocked` and `quality_checks_failed > 0`. No `MetricResult` rows are written for that run.

Q23: What types of quality checks does MetaSignal implement?

Three types in v0: `required_columns_present` (column list match), `user_id_null_rate` (null fraction against threshold), `event_type_domain_check` (allowed values). Two additional checks also pass on clean data (exact names not read from code, but confirmed via `checks_passed: 5`).

Q24: What does failure injection prove?

That the quality gates actually block computation, not just log a warning. Each of the 3 injected scenarios produces `expected_behavior_met: true` — the system did what it was supposed to do when given bad data.

Q25: Why 3 failure scenarios specifically?

They cover the three most common real-world data quality failure modes: schema changes (missing column), bad values (invalid event type), and null spikes (data pipeline issues). A real system would have dozens more.

Category 6: Experiment Assignment / SRM

Q26: Why deterministic assignment instead of random?

Reproducibility. If you use random assignment with a timestamp seed, you can't reconstruct who was in which variant after the fact without storing every assignment. With deterministic hash, you can always recompute the assignment from (`experiment_key`, `user_id`). No variant lookup table needed.

Q27: How does the hash assignment work exactly?

`SHA-256("{experiment_key}:{user_id}")`. Take the first 12 hex characters. Parse as integer. Divide by 16^{12} . This gives a bucket in $[0,1)$. If `bucket < treatment_split_pct`, the user is in treatment.

Q28: Does using the same hash function for all experiments mean users always get the same variant?

No. The `experiment_key` is part of the input to SHA-256. Different experiment keys produce different hashes for the same `user_id`. A user who was in treatment for experiment A may be in control for experiment B.

Q29: What is the SRM threshold and why?

$p < 0.001$. SRM testing is a quality check, not a scientific hypothesis test. You use a stricter threshold to avoid discarding valid experiments — a $p=0.01$ threshold would incorrectly invalidate 1% of clean experiments.

Q30: What does `balance_status: pass` mean in the assignment balance artifact?

The absolute imbalance between observed treatment share (50.47%) and expected (50%) is 0.47 percentage points, below the 1 pp threshold. The assignment is considered balanced.

Category 7: CUPED / A/A Validation

Q31: Explain CUPED in 30 seconds.

CUPED reduces experiment noise by subtracting out the part of the outcome that was predictable from pre-experiment behavior. Mathematically: compute $\theta = \text{Cov}(Y_{\text{post}}, X_{\text{pre}}) / \text{Var}(X_{\text{pre}})$. Subtract $\theta \times (X_{\text{pre}} - \text{mean}(X_{\text{pre}}))$ from each user's outcome. The adjustment is mean-zero so the expected treatment effect is unchanged, but variance decreases by ρ^2 .

Q32: Why does variance reduction help?

Lower variance $\rightarrow$ smaller standard error $\rightarrow$ narrower confidence interval $\rightarrow$ smaller p-value for the same sample size. Or equivalently: same confidence with smaller sample. Running experiments for less time or with fewer users.

Q33: Why is variance reduction only 0.44% in the MetaSignal experiment?

Pre-event count is a weak predictor of binary purchase conversion over 14 days. The covariate correlation (ρ) is near zero, so $\rho^2 \approx 0$. CUPED is essentially doing nothing. A stronger covariate — like prior purchase rate or prior revenue — would give much higher reduction.

Q34: What is the KS test checking?

Whether the CUPED p-value distribution under the null (A/A test) is $\text{Uniform}(0,1)$. If CUPED distorted the null distribution — inflating p-values downward or upward — the KS statistic would be large. KS statistic = 0.019 vs critical value = 0.043 $\rightarrow$ passes. CUPED doesn't break the null distribution.

Q35: What does the strong covariate edge case show?

With $r \approx 0.75$ (strong correlation between X_{pre} and Y_{post}), CUPED achieves 50.7% variance reduction. This shows the algorithm can produce large reductions when the covariate is appropriate.

Q36: What happens when there's no pre-period data?

`cuped_edge_case=missing_xpre`, `cuped_applied=false`, `cuped_fallback=unadjusted`. The system gracefully falls back to the raw estimator rather than crashing or silently producing a bad adjustment.

Q37: Could CUPED make variance worse?

In principle, yes — if ρ is very small, the θ estimate has noise that can add more variance than it removes. That's why the `weak_covariate` edge case falls back to unadjusted (threshold $|\theta| < 0.05$). A good implementation never makes things worse.

Category 8: Guardrails / Right-Censoring

Q38: What's the difference between a guardrail and a primary metric?

The primary metric is the outcome you're trying to improve. The guardrail is a harm indicator you're trying to keep below a threshold. The primary metric drives the hypothesis; the guardrail blocks shipment if harm is too high.

Q39: Why isn't the guardrail just another hypothesis test with Bonferroni correction?

Guardrails are not about detecting a statistically significant effect — they're about checking whether a business tolerance is breached. The 2% refund rate tolerance is a business decision, not a statistical threshold. Bonferroni correction applies when you're testing multiple hypotheses for discovery; guardrails are specification tests, not discovery tests.

Q40: Why does a statistically positive primary metric get HOLD?

Because product harm is asymmetric. A 420% lift in conversion with a 3.1% refund rate increase means you're growing revenue at the cost of customer trust and operational overhead. Guardrail-first means the harm constraint takes priority.

Q41: What is right-censoring in this context?

The refund rate guardrail has a maturity window of 30 days. Reading it at day 14 means 16 days of refund data is still missing — the early reading is incomplete (right-censored). The system marks it `right_censoring_flag=true`, `is_mature=false`, to prevent a premature final decision based on an incomplete guardrail read.

Q42: What does the override mechanism demonstrate?

That the system supports human override with accountability. If a PM wants to ship despite a guardrail breach, they can — but they must provide a written reason. That reason is stored in the `DecisionLog.override_reason` field. The API enforces this at HTTP 422.

Q43: What would happen if the API's override enforcement were bypassed by writing directly to the DB?

The DB has no constraint enforcing `override_reason` non-null when decisions differ. The enforcement is at the API layer only. A direct DB write could insert a row with `override_reason=null` and `final_decision != system_recommendation`. This is a gap — acknowledged in Section 10 (L8).

Category 9: Anomaly Detection

Q44: Why DOW adjustment?

E-commerce metrics have strong weekly seasonality. A Monday with 20% above baseline isn't anomalous if every Monday is 20% above baseline. DOW adjustment compares the current Monday against previous Mondays, eliminating the weekly pattern from the baseline.

Q45: What happens when there aren't enough same-DOW historical points?

Falls back to a 14-day rolling window. This is less accurate but avoids crashing when there's limited history.

Q46: What's the anomaly threshold and why 3.0?

$z \geq 3.0$. This is a standard choice that gives approximately 0.27% false alarm rate under a normal distribution. More aggressive thresholds ($z=2.0$) fire too frequently. More conservative ($z=4.0$) miss real anomalies.

Q47: What does the backtest prove specifically?

That DOW adjustment suppresses 3 false positives (known DOW events that raw z-score fires on) while still detecting all genuine anomalies and delayed detections (events where $DOW\ z > raw\ z$). The 20-event labeled set covers 5 categories of behavior.

Category 10: Causal Traps

Q48: What is the peeking problem?

Looking at experiment results before the experiment is complete and stopping early when you first see a "significant" result. This inflates the actual false positive rate. If you peek 10 times, each at $p < 0.05$, your true false positive rate can reach 40%.

Q49: What does MetaSignal do about peeking?

It flags `peeking_warning=true` when significance conclusions change across interim reads. It doesn't apply sequential testing corrections — that would require implementing always-valid confidence intervals or SPRT, which are next steps.

Q50: What is a novelty effect?

Users engage more with new features simply because they're new. This inflates week-1 metrics. If you ship based on week-1 lift, you may be shipping based on curiosity, not real value.

Q51: How does MetaSignal flag novelty?

`novelty_flag = (week_1_lift / week_2_lift) > 2.0`. If week-1 lift is more than twice week-2 lift, the flag fires. 7x ratio in the simulated data.

Q52: What is Simpson's paradox in this context?

The overall positive lift (+3.1%) masks negative lifts in specific segments (India: -1.2%, iOS: -1.8%). Shipping based on the overall result would harm Indian and iOS users. The `simpsons_flag` catches this.

Category 11: Streaming

Q53: Why build a streaming layer for a batch system?

To demonstrate that I understand where streaming fits in an experimentation stack. Streaming is not a replacement for batch — it's an early warning layer. You want to know if SRM is developing before you wait 24 hours for the batch job. You want to know if a platform's events stopped flowing before the daily report. Streaming gives you that at the cost of provisionality.

Q54: Why not Kafka?

Kafka requires infrastructure I didn't want to set up for a portfolio project. The `EventSource` interface is designed to be swappable. The consumer, validator, monitors, and reconciler all work against that interface. Adding `KafkaEventSource(EventSource)` is a bounded, well-defined extension.

Q55: What does DLQ mean and why does it matter?

Dead Letter Queue. Events that fail validation (missing required fields, invalid variant, non-numeric value) are quarantined in the DLQ instead of being silently dropped. This preserves them for debugging. The DLQ in MetaSignal writes `dlq_events.json` and `dlq_events.parquet` with error codes per event.

Q56: What is stream-batch reconciliation?

Comparing the streaming provisional metric value against the batch authoritative value after the batch job runs. The delta tells you how much to distrust the streaming number. In MetaSignal, the simulated delta is 6% (batch = 94% of streaming). The reconciliation report marks `batch_remains_authoritative=true`.

Q57: What are the 10 streaming scenarios?

SRM assignment drift (windows 2-4), iOS event flow gap (windows 5-6), late-arriving events (window 8), duplicate events (window 10), malformed events — DLQ poisoning (window 4), consumer lag spike (window 7), provisional anomaly spike (window 9), stream-batch mismatch (all windows), DLQ type mismatch (window 4), and replay recovery (all windows).

Q58: How does the streaming SRM detection work differently from batch SRM?

Batch SRM uses all assignments from the DB. Streaming SRM uses rolling 6-window slices of event traffic. It fires when the observed control percentage deviates >5 pp (investigate) or >2 pp (warn) from the expected 50%. Streaming SRM is provisional and must be confirmed by batch — which is why each alert has `provisional_status: pending_batch_confirmation` and `batch_confirmation_status: confirmed`.

Category 12: Stream-Batch Reconciliation

Q59: What causes streaming to differ from batch?

Four sources: late events (not yet arrived in streaming window), duplicate events (streaming counts twice, batch deduplicates), DLQ events (streaming quarantined them, batch may include them), and CUPED adjustment (batch applies CUPED, streaming doesn't).

Q60: How big is the streaming-batch delta in MetaSignal?

6% (streaming is 1/0.94 times batch authoritative). This is simulated — the exact 0.94 factor is hardcoded in `StreamBatchReconciler.reconcile()`.

Category 13: API / CI / Docs

Q61: What FastAPI endpoints exist?

GET `/health` (DB counts), GET `/metrics` (metric list), GET `/metrics/{name}/timeseries`, GET `/experiments` (list), GET `/experiments/{key}/readout`, GET `/data-quality/latest`, GET `/evidence/{artifact_name}`, GET `/validation/{artifact_name}`, GET `/resume-signal`, POST `/decisions/validate-override`.

Q62: What does `/resume-signal` return?

The content of `outputs/reports/metasignal_resume_signal_summary.json` — 10 capability signals, each with proof, artifact, and interview signal text.

Q63: What does the GitHub Actions CI test?

On push to main/metasignal-prd-core and on PRs: run `run_prd_core_validation_suite_v1.py` (CUPED edge cases, golden scenarios, causal traps, anomaly backtest) and `show_streaming_demo.py` + `validate_streaming_prd_v1.py` (43 streaming checks). No DB in CI.

Q64: What's missing from CI?

A PostgreSQL service container. All scripts that require a live DB (CUPED, SRM, guardrail decision, metric compute) are not CI-gated. CI catches statistical logic and streaming validation. A full CI would add the DB-dependent scripts.

Category 14: Limitations / Red-Team

Q65: What would break at real scale?

Many things. The deterministic assignment stores 20,000 rows per experiment — at 10M daily active users, that's 10M rows per experiment per day. The in-memory streaming queue wouldn't hold real event volumes. The current anomaly detection runs serially over all metric rows — would need partitioning. No horizontal scaling, no distributed compute.

Q66: What's the biggest gap between MetaSignal and a real experimentation platform?

Real users and real assignment infrastructure. MetaSignal assigns users from a parquet file — a real platform assigns users at request time, at the product surface. The assignment call must be sub-millisecond, globally consistent, and integrated into every product surface. That's the hardest part of experimentation at scale, and it's absent here.

Q67: Is MetaSignal production-ready?

No. It's a well-structured local prototype. "Production-simulated" means it handles realistic failure modes and produces evidence — not that it would survive production traffic.

Category 15: Resume / Positioning

Q68: How would you describe MetaSignal on your resume?

"Built MetaSignal: a production-simulated experimentation intelligence platform with versioned metric registry, denominator conflict detection, data quality gates, deterministic SHA-256 assignment, SRM checks, CUPED with A/A validation, guardrail-first decisioning, right-censoring, DOW-adjusted anomaly detection, causal trap flags, and asyncio-simulated streaming layer. 50+ JSON evidence artifacts."

Q69: Who is the target audience for MetaSignal?

Hiring managers for experimentation platform, data engineering, analytics engineering, or senior data science roles at product companies that run A/B tests at scale.

Q70: What does MetaSignal signal about you to an interviewer?

That you think about data infrastructure, not just statistical models. That you understand the full lifecycle from data quality to final decisions. That you're honest about what you built and what you didn't. That you can produce verifiable, reproducible evidence.

Q71: What's the weakest part of your portfolio positioning with MetaSignal?

The absence of real user data and real assignment infrastructure. The treatment effect is synthetic. The guardrail breach is simulated. These are honest limitations but they're genuine gaps. Be upfront — interviewers respect honesty more than inflated claims.

Q72: Which companies would find MetaSignal most relevant?

Any tech company with A/B testing at scale: product analytics teams, experimentation platform teams, growth analytics teams. Roles: experimentation engineer, analytics engineer, senior data scientist with experimentation focus.

Q73: What follow-up questions should you anticipate after showing MetaSignal?

"Is this deployed anywhere?", "What's the treatment effect?", "Why is CUPED variance reduction so low?", "Did real analysts use this?", "Why not Kafka?", "What would break at scale?"

Additional questions (Q74–Q84)

Q74: How do you handle the case where a user is in both treatment and control across different experiments?

The `experiment_key` namespace in the SHA-256 assignment ensures different variants per experiment. User A might be treatment in experiment X and control in experiment Y. This is the standard multi-experiment approach — each experiment is independent.

Q75: What is the `evaluator_version` field?

Stored in `ExperimentEvaluation`. Currently `cuped_v0`. Designed for versioning the evaluation logic — when the CUPED implementation changes, the version bumps and old evaluations remain linked to the old evaluator.

Q76: What does the `segment_analysis JSONB` field contain?

In the current state, it stores: covariate type, synthetic uplift user count, and a note about the evaluation approach. In a more complete implementation, it would store segment-level lift estimates per country, platform, etc.

Q77: Can a single experiment have multiple guardrail metrics?

Yes. `Experiment.guardrail_metric_configs` is a JSONB field designed to hold multiple guardrail definitions. `ExperimentEvaluation.guardrail_results` is also JSONB, storing results for each guardrail. The current experiment uses one guardrail, but the schema supports multiple.

Q78: What is the `is_aa_test` field?

A Boolean in `ExperimentEvaluation`. Marks whether the evaluation row was produced by an A/A test. A/A test evaluations should show non-significant results — if they show significant results, that indicates a calibration problem. Currently `false` in live DB.

Q79: What's the difference between `run_full_demo_v0.py` and `run_metasignal_prd_complete_v1.py`?

`run_full_demo_v0.py` runs all 24 v0 scripts in sequence — the full data pipeline, core experiments, and exports. `run_metasignal_prd_complete_v1.py` runs the v1 PRD completion suite: seed operational history, v1 core validations (CUPED edge cases, golden scenarios v1, causal traps, anomaly backtest), override enforcement test, streaming demo, and streaming validation. They serve different purposes — v0 is the core demo, v1 is the PRD completion evidence.

Q80: How many total quality check rows are in the DB?

At least 30 failed rows (from failure injection). More passing rows from clean pipeline runs. The golden scenario suite confirms `quality_failures_captured` scenario observes 30 failed checks.

Q81: What is the `guardrail_window_days` field?

Part of `MetricDefinition`. Specifies how many days the guardrail metric needs to mature before it can be considered for a final ship decision. `refund_rate_guardrail` has `guardrail_window_days=30`. This feeds the right-censoring logic in `run_post_launch_validation_v0.py`.

Q82: What does `triggered_by` in `PipelineRunLog` mean?

The name of the script or process that initiated the pipeline run. E.g., `compute_metrics_v0`, `failure_injection_v0`. This is for traceability — when debugging an anomaly, you can look up which job produced the metric and who triggered it.

Q83: What does `experiment.created_by` store?

The name/identifier of the person or system that created the experiment record. In the current seed, it's the seeder script name. In a real system, it would be a user ID or email.

Q84: If this project had one more week of development, what would you add?

PostgreSQL service container in CI (closes the biggest CI gap), a better CUPED covariate (prior purchase rate), proper sequential testing in peeking detection (always-valid CIs), and a basic API-connected data visualization for the `/resume-signal` endpoint. In that priority order.

Section 12 — Cross-Examination Section

The Hostile Interviewer

The following questions are designed to be adversarial. Strong answers are given for each.

"Isn't this just a bunch of scripts?"

"No. It's a system. Each script has a defined interface — it reads from DB or file, it writes to DB or file, it produces an evidence artifact. The 9-table DB schema is migrated and versioned. The API serves the artifacts via HTTP. The CI validates the logic on every push. A bunch of scripts doesn't have a migration history, an API layer, a streaming architecture with 15 classes, and a validation suite that runs 43 checks. What I've built has all of those."

"Why should I believe this is production-simulated?"

"Because every behavioral claim has a verification path. The guardrail-first behavior is not just claimed in a README — it's demonstrated by `guardrail_decision_report.json` with `guardrails_cleared=false` and `final_decision=HOLD` despite `p=5e-44`. The SRM check doesn't just exist in code — `srn_check_report.json` shows the chi-square statistic and p-value. The streaming scenarios don't just run — `streaming_prd_scenario_report.json` shows each scenario returning `implemented=true`. You can verify every claim by reading the artifact."

"Where is the real production traffic?"

"There isn't any. MetaSignal is production-simulated, not production. The RetailRocket dataset is 2.75 million real e-commerce events, but from a public dataset, not from a live system. I don't claim live traffic and I don't need to — the purpose is to demonstrate the infrastructure design and statistical discipline, not the scale."

"Why is the treatment effect synthetic?"

"Because RetailRocket has no embedded A/B experiment. I needed a non-null readout to demonstrate the evaluation pipeline end-to-end, so I deterministically injected 255 treatment conversions. The injection is documented in the code and in the artifact (`synthetic_uplift_users: 255`). The computation is real — the assignment, CUPED, z-test, DB row, API response. The effect source is synthetic. That distinction matters and I make it clearly."

"Why is CUPED variance reduction only 0.44%?"

"Because pre-event count has very low correlation with binary purchase conversion. CUPED's effectiveness is ρ^2 — if $\rho \approx 0$, you get near-zero reduction. That's honest. If I had used prior purchase rate as the covariate, I'd get much higher reduction. I used `pre_event_count` because it's what's available in RetailRocket and it demonstrates the algorithm — the A/A validation with synthetic $r=0.6$ shows the algorithm can achieve 26.5% reduction when the covariate is appropriately chosen."

"Why did you inject uplift?"

"To produce a non-trivial readout for the guardrail demonstration. If the primary metric showed zero lift, the guardrail-first behavior — HOLD despite positive primary — couldn't be demonstrated. I needed a positive primary metric to show that even a very positive result gets blocked by the guardrail. The injection is designed to serve that purpose, and it's fully documented."

"Doesn't 420% lift look fake?"

"Yes, and it is. I make that explicit in every artifact and every document. The control conversion rate is 0.65% (realistic for a 14-day window on RetailRocket). 255 injected conversions on top of ~10,094 treatment users raises treatment conversion to 3.4%. The ratio is $3.4/0.65 \approx 5.2\times$ — large but not impossible for a strong intervention. The key point is: the lift is from injection, not from a real experiment effect. I document that clearly."

"Did you actually implement CUPED or just write about it?"

"I implemented it. `run_cuped_v0.py` computes θ via `Cov(Y_post, X_pre) / Var(X_pre)`. It applies the adjustment. It runs a two-sided z-test on both raw and CUPED outcomes. It writes an ExperimentEvaluation row to the DB with `cuped_applied=true, cuped_theta=0.00378, variance_reduction_pct=0.44`. I validated it on 1,000 synthetic A/A runs — false positive rate 5.5%, KS test passes. I tested 5 edge cases — weak covariate, missing pre-period, zero variance, imbalanced arms. The implementation is real and verifiable."

"Are your anomaly precision/recall numbers real?"

"No — and I say so in the artifact. `synthetic_labeled: true, claim_boundary: 'Precision/recall measured on synthetic injected anomaly events only; not production precision/recall.'` The 20 labeled events are hand-crafted. The numbers (precision=1.0, recall=0.87) demonstrate that the algorithm behaves correctly on known cases. They are not production metrics."

"Did real analysts make decisions?"

"No. There are no real analysts. The decision log has two HOLD decisions — one by 'Demo Reviewer', one by 'Guardrail Engine v0'. Both are seeded by scripts. This is a solo portfolio project — the decision workflow demonstrates design, not operational use."

"Why no Kafka?"

"Kafka requires infrastructure I chose not to set up for a portfolio project. The streaming layer uses asyncio simulation but is designed around an `EventSource` abstract class. Adding `KafkaEventSource(EventSource)` that consumes from a Kafka topic would leave the consumer, validator, and all monitors unchanged. The interface separation is deliberate. I can

explain the design and what Kafka would add (real consumer group semantics, at-least-once delivery, replay from offset) without needing Kafka running."

"Why no Spark?"

"The dataset is 2.75 million rows. Pandas handles it comfortably on a single machine. For a portfolio project demonstrating statistical infrastructure, adding Spark would be complexity for its own sake. At real scale — tens of billions of events — you'd move metric computation to Spark or Trino. That's an obvious extension, not a design flaw."

"Why no frontend?"

"Building a frontend was out of scope. The API layer is the interface — it exposes DB state and evidence artifacts via HTTP. A frontend would consume the API. The `/resume-signal` endpoint exists specifically as the starting point for a resume/portfolio page. I chose to invest the time in correctness of the backend logic rather than a UI."

"Why does CI not run the full DB pipeline?"

"The GitHub Actions workflow uses lightweight dependencies — `numpy fastapi httpx pandas pyarrow sqlalchemy pg8000`. It doesn't configure a PostgreSQL service. Scripts requiring live DB (CUPED, SRM, guardrail decision) are not CI-gated. I acknowledge this as a gap. Adding a `services: postgres:` block to the workflow and running `run_full_demo_v0.py` would close it. I prioritized getting the statistical validation and streaming validation into CI first."

"Are all tables migrated?"

"Nine tables are Alembic-migrated: the 8 from migration 1 (`metric_definitions`, `metric_results`, `pipeline_run_log`, `data_quality_checks`, `simulation_events`, `experiments`, `experiment_evaluations`, `decision_log`) and `experiment_assignments` from migration 2. Four support tables (`metric_conflicts`, `backfill_jobs`, `anomaly_alerts`, `system_health_reports`) are created by the operational history seeder via raw SQL — they're for simulation scenarios and would be Alembic-migrated in a production system."

"What would break at real scale?"

"Five things immediately: (1) In-memory streaming queue wouldn't hold real event volumes. (2) Deterministic assignment stored in DB — 10M users × N experiments per day = enormous assignment table. Real systems would compute assignments at request time, not store them. (3) Metric computation runs serially in Python — at scale, you'd use distributed compute (Spark/Trino). (4) No read replicas or caching — the API would fall over under real query load. (5) Single-node DB — needs replication and failover. These are all known gaps, not oversights."

"What would you do next?"

"In priority order: (1) PostgreSQL in CI. (2) Better CUPED covariate — use prior purchase rate, which is available in RetailRocket. (3) Sequential testing (always-valid CIs) to actually fix the peeking problem rather than just flagging it. (4) Kafka event source implementation — `KafkaEventSource(EventSource)` is a bounded extension. (5) A minimal frontend consuming the API — start with the `/resume-signal` endpoint. (6) Real experiment data — find a dataset with an embedded A/B experiment, or generate one with explicit DGP and controlled effect size."

End of Part 2 (Sections 7–12). Continue in Part 3 (Sections 13–17) for Resume Claim Audit, Do Not Say This, Study Plan, Cheat Sheet, and Final Verdict.

SECTION 13: RESUME CLAIM AUDIT

How to use this section: Each bullet is rated for defensibility. "Strong" = back it up cold. "Safer" = use if you can't go deep. "Ultra-conservative" = minimum viable claim. Pick based on your interviewer's likely depth.

13.1 SIX STRONG RESUME BULLETS

BULLET S1 — Core Architecture

Text:

"Designed and implemented a batch-authoritative experimentation intelligence platform in Python/PostgreSQL, covering the full A/B testing lifecycle: user assignment (SHA-256 deterministic hashing), SRM detection (chi-square, $p < 0.001$ threshold), CUPED variance reduction, guardrail-first decision logic, and causal trap detection."

Defensibility Rating: 9/10 — every component exists in code and has artifact output.

Evidence Paths:

- `scripts/generate_assignments_v0.py` → SHA-256 deterministic hashing
- `scripts/run_srm_check_v0.py` → chi-square SRM, threshold $p < 0.001$
- `scripts/run_cuped_v0.py` → CUPED implementation
- `scripts/run_guardrail_decision_v0.py` → guardrail + decision log
- `scripts/run_prd_core_validation_suite_v1.py` → causal trap detectors

Risky Words to Avoid: "production," "deployed," "serving traffic"

Safer Wording: Add "in a production-simulated environment using the RetailRocket public dataset" to contextualize.

Strongest Version: As written above — fully defensible if you know all five components.

BULLET S2 — Statistical Rigor

Text:

"Implemented CUPED covariate adjustment with five edge-case handlers (strong correlation, weak correlation fallback, missing pre-period fallback, zero-variance fallback, imbalanced arms); validated via 1,000 synthetic A/A runs achieving 5.5% FPR and KS test pass ($stat=0.019$ vs $critical=0.043$)."

Defensibility Rating: 10/10 — all numbers sourced directly from artifact JSON.

Evidence Paths:

- `outputs/validation/cuped_edge_case_results.json` → 5 edge cases, VR percentages
- `outputs/validation/cuped_aa_validation_results.json` → 1000 runs, FPR=5.5%, KS $stat=0.019$
- `scripts/run_cuped_aa_validation_v0.py` → implementation
- `scripts/run_prd_core_validation_suite_v1.py` → `apply_cuped_with_edge_cases()`

Risky Words to Avoid: "production data," "real users," "deployed CUPED"

Safer Wording: "Validated statistical correctness via synthetic simulation" is an acceptable hedge.

Strongest Version: As written — the numbers are exact and documented.

BULLET S3 — Streaming Extension

Text:

"Built an asyncio-based streaming simulation layer with 15 classes handling event ingestion, dead-letter queuing (3 error codes), SRM detection, traffic monitoring, stream-batch reconciliation, and replay management; validated 43/43 PRD checks passing across 10 failure injection scenarios."

Defensibility Rating: 8/10 — all 15 classes exist; "streaming" terminology requires the asyncio caveat.

Evidence Paths:

- `src/metasignal/streaming/streaming_prd_v1.py` → all 15 classes
- `outputs/validation/streaming_prd_v1_validation.json` → 43/43
- `outputs/streaming/scenario_report.json` → 10 scenarios
- `outputs/streaming/dlq_events.json` → 3 DLQ error codes

Risky Words to Avoid: "Kafka," "real-time production streaming," "distributed streaming"

Safer Wording: "asyncio-based event simulation" or "streaming-pattern implementation."

Strongest Version: Replace "streaming simulation layer" with "streaming-pattern pipeline" — still accurate, less provocative.

BULLET S4 — Operational Intelligence

Text:

"Seeded a 60-day operational history across 13 simulation scenarios covering schema drift, null spikes, late-arriving data, human overrides, and clean ship decisions; implemented DOW-adjusted z-score anomaly detection with backtest precision=1.0, recall=0.87."

Defensibility Rating: 9/10 — all 13 scenarios named in seeder; backtest numbers in artifact.

Evidence Paths:

- `scripts/seed_operational_history_v1.py` → all 13 SIM scenarios
- `outputs/evidence/anomaly_detection_results.json` → precision, recall
- `scripts/run_anomaly_detector_v0.py` → DOW-adjusted z-score logic

Risky Words to Avoid: "production monitoring," "live anomalies detected"

Safer Wording: "Designed and implemented" is accurate.

Strongest Version: As written — "simulation scenarios" already contextualizes the data.

BULLET S5 — API and Validation Infrastructure

Text:

"Exposed platform capabilities via 10 FastAPI endpoints (smoke-tested 8/8); enforced override governance with HTTP 422 rejection of undocumented overrides; validated end-to-end system completeness against a 34-item PRD checklist (13 core + 21 streaming items, all passing)."

Defensibility Rating: 9/10 — all counts verified in code and artifacts.

Evidence Paths:

- `src/metasignal/api/app.py` → 10 endpoints
- `scripts/smoke_test_api_v0.py` → 8/8 pass
- `scripts/test_override_enforcement_v1.py` → HTTP 422
- `outputs/reports/metasignal_prd_completion_report_v1.json` → 34 items

Risky Words to Avoid: "production API," "serving external users"

Safer Wording: "Exposed and validated" vs "deployed" — stick with "exposed."

Strongest Version: As written — counts are exact and auditable.

BULLET S6 — Database and Schema

Text:

"Designed a 13-table PostgreSQL schema (9 tables via Alembic migrations, 4 via seeder raw SQL) with SQLAlchemy ORM models covering experiment lifecycle, metric definitions, decision audit trails, and operational health; used 2,756,101 rows from the public RetailRocket dataset as the behavioral foundation."

Defensibility Rating: 9/10 — table counts verifiable in migrations + seeder.

Evidence Paths:

- `alembic/versions/61738c5dba04_*.py` → 8 tables in first migration
- `alembic/versions/de8aedf29d09_*.py` → `experiment_assignments`
- `scripts/seed_operational_history_v1.py` → 4 support tables via raw SQL
- `src/metasignal/db/models.py` → ORM definitions

Risky Words to Avoid: "production database," "transactional system"

Safer Wording: "PostgreSQL schema for a production-simulated experimentation platform"

Strongest Version: As written — 13 = 9 + 4 is verifiable and defensible.

13.2 SIX SAFER RESUME BULLETS

Use these if you cannot go deep on specific technical numbers.

BULLET SF1

"Built an end-to-end A/B testing platform in Python and PostgreSQL covering user assignment, statistical analysis, and automated decision logic."

- **Defensibility:** 10/10 — extremely conservative, impossible to challenge
- **Trade-off:** Loses differentiation signal

BULLET SF2

"Implemented CUPED variance reduction and SRM detection for experiment quality assurance, with automated edge case handling."

- **Defensibility:** 10/10
- **Trade-off:** No numbers, lower impact

BULLET SF3

"Designed streaming-pattern event processing architecture with dead-letter queuing, reconciliation logic, and failure injection testing."

- **Defensibility:** 10/10 — "streaming-pattern" removes Kafka confusion
- **Trade-off:** Slightly less impressive sounding

BULLET SF4

"Created a 60-day operational simulation seeding realistic failure scenarios including schema drift, data quality degradation, and human override governance."

- **Defensibility:** 10/10
- **Trade-off:** Sounds like data engineering, less stats-forward

BULLET SF5

"Validated platform correctness against a comprehensive PRD checklist and implemented CI with GitHub Actions for automated regression testing."

- **Defensibility:** 10/10
- **Trade-off:** CI-focused bullets are supporting detail, not headline claims

BULLET SF6

"Applied causal trap detection for peeking bias, novelty effect, and Simpson's paradox to protect experiment integrity."

- **Defensibility:** 10/10 — all three detectors exist in `run_prd_core_validation_suite_v1.py`
- **Trade-off:** No numbers attached, but the concepts are the signal

13.3 THREE ULTRA-CONSERVATIVE BULLETS

Use only if explicitly asked for minimal/simple descriptions.

UC1: *"Built a solo Python project demonstrating A/B testing concepts including statistical analysis, database design, and API development."*

UC2: *"Implemented statistical methods (CUPED, SRM detection, z-score anomaly detection) in a self-contained experimentation system using public retail data."*

UC3: *"Designed and coded an experimentation platform with PostgreSQL backend, FastAPI interface, and asyncio-based event simulation."*

13.4 AUDIENCE-SPECIFIC SUMMARIES

LinkedIn Project Summary

MetaSignal is a batch-authoritative experimentation intelligence platform I built solo to deeply understand A/B testing infrastructure. It covers the full experiment lifecycle: deterministic user assignment (SHA-256), SRM detection, CUPED variance reduction with 5 edge cases, guardrail-first decision logic, causal trap detection, and operational anomaly monitoring. The platform includes a 13-table PostgreSQL schema, 10 FastAPI endpoints, an asyncio streaming simulation layer, and a 60-day seeded operational history across 13 failure scenarios. Built on the public RetailRocket dataset (2.7M rows). All capabilities validated against a 34-item PRD checklist.

GitHub Project Summary (README tagline)

Production-simulated A/B testing platform — full experiment lifecycle in Python/PostgreSQL. CUPED + SRM + guardrails + causal traps + streaming simulation. 2.7M rows, 43/43 PRD checks passing.

Recruiter Explanation (30 seconds)

"I built a complete experimentation platform from scratch — think of it like a simplified internal tool that companies like Meta or Airbnb use to run A/B tests. It handles assigning users to experiments, measuring results statistically, detecting data quality issues, and making automated decisions about whether to ship a feature. I used real public data and validated everything against a detailed spec. The goal was to understand the full system, not just the statistics."

Hiring Manager Explanation (60 seconds)

"MetaSignal is a solo project I built to go from 'I understand A/B testing' to 'I can build and reason about the full system.' It covers user assignment with deterministic hashing, statistical analysis including CUPED variance reduction and SRM detection, guardrail-first decision logic that blocks ships when safety metrics breach tolerance, and an anomaly detection layer. I also simulated a streaming event processing pipeline and seeded 60 days of operational history including failure scenarios like schema drift and data quality events. It's built on PostgreSQL with SQLAlchemy ORM and exposed via FastAPI. Everything is validated against a PRD I wrote. Not deployed to production — but designed with production patterns in mind."

Technical Interviewer Explanation (2 minutes)

"MetaSignal is a batch-authoritative experimentation platform. The core decision is: streaming data is provisional and never authoritative — final experiment evaluations run against the batch-recomputed database state. For assignment, I use SHA-256 deterministic hashing: $\text{bucket} = \text{int}(\text{SHA256}(\text{'expkey:userid'})[:12], 16) / 16^{12}$. SRM detection is chi-square goodness-of-fit with $\text{df}=1$, threshold $p<0.001$. CUPED uses $Y_{\text{adj}} = Y_{\text{post}} - \theta \times (X_{\text{pre}} - \text{mean}(X_{\text{pre}}))$ where $\theta = \text{Cov}(Y,X)/\text{Var}(X)$, with five fallback handlers. The guardrail logic blocks ship decisions when a safety metric — refund rate in my example — exceeds tolerance. I seeded 13 operational scenarios including a real anomaly (iOS outage), a DOW false positive, and a human override. The streaming layer uses asyncio with 15 classes, including dead-letter queuing with three error codes and stream-batch reconciliation. Not Kafka — I want to be clear about that. 43/43 streaming PRD checks pass, 13/13 core PRD items pass."

SECTION 14: DO NOT SAY THIS

Hard rule: If you say any of the following without the specified qualifier, you have overclaimed. Interviewers at Meta/Google/Stripe know the difference.

14.1 ABSOLUTE DO NOT SAY (No Qualifier Saves These)

Do Not Say	Why It's Wrong	What to Say Instead
"MetaSignal runs in production"	It has never served live traffic	"MetaSignal is production-simulated, not deployed"
"I used Kafka for streaming"	asyncio only, no Kafka anywhere	"I built an asyncio-based streaming simulation"
"Real users ran through this"	Synthetic + RetailRocket only	"I used the public RetailRocket dataset and synthetic users"
"The experiments generated business impact"	No business outcome was measured	"The system architecture is designed to support business decisions"
"I deployed this to AWS/GCP/Azure"	No cloud deployment exists	"I ran it locally with Docker/PostgreSQL"

Do Not Say	Why It's Wrong	What to Say Instead
"This is how Meta does it"	You don't know Meta's internals	"This mirrors published patterns from Trustworthy Online Experiments"
"I trained a model on the results"	No ML model exists in this repo	"I implemented statistical decision logic — no ML component"

14.2 HIGH-RISK PHRASES (Need an Immediate Qualifier)

High-Risk Phrase	What Interviewers Hear	Safe Version
"real-time streaming"	Kafka, Flink, actual low-latency	"asyncio-based event simulation that mimics real-time patterns"
"live data pipeline"	Production data flowing	"seeded operational history simulating a live pipeline"
"anomalies detected"	Production monitoring found issues	"synthetic anomaly scenarios injected and correctly flagged"
"7 metrics"	vs "3 metrics" contradiction	"3 core metrics in the base DB, 7 after seeding operational history"
"the system scales to..."	Implies load-tested at scale	"designed with scale patterns in mind; not load-tested"
"I monitored experiment health"	Implies active monitoring job	"I built the monitoring components and validated them via simulation"
"the API serves requests"	Implies live traffic	"the API was smoke-tested locally with TestClient"
"I validated the results"	Implies ground truth comparison	"the system's outputs were validated against predefined PRD assertions"

14.3 SUBTLE OVERCLAIMS IN INTERVIEW ANSWERS

On CUPED:

- ■ "CUPED reduced variance by 50% in my experiments"
- ✓ "In the strong-correlation edge case, the simulated VR was 50.7%. The actual experiment showed 0.44% VR, which is expected given the synthetic data."

On streaming:

- ■ "My streaming system processes events in milliseconds"
- ✓ "My asyncio simulation doesn't measure real latency — it's designed to exercise the logic paths, not benchmark throughput."

On A/A testing:

- ■ "My A/A test confirmed no bias in the system"

- ✓ "The A/A simulation across 1,000 synthetic runs produced a 5.5% FPR — within acceptable range — and the KS test on p-value distribution passed. That validates the statistical implementation, not that the system is bias-free in a production sense."

On the decision to ship:

- ■ "The algorithm decided to ship the feature"
- ✓ "The HOLD decision was triggered because the refund guardrail exceeded the 2.0% tolerance threshold. The system surfaces the recommendation; a human would make the final call."

On dataset size:

- ■ "I processed 2.7 million rows of user data"
- ✓ "I loaded 2.7 million rows from the public RetailRocket dataset into PostgreSQL for behavioral simulation."

14.4 WORDS THAT INVITE FOLLOW-UP YOU MIGHT NOT WANT

Word	What It Invites	Your Exit Strategy If Asked
"production-grade"	"Show me your SLOs, rollback plan, and incident runbook"	"Production-patterned — I followed production design principles without deploying"
"optimized"	"What were the before/after benchmarks?"	Avoid unless you measured something specific
"scalable"	"How do you scale it? What's the bottleneck?"	"Designed with stateless assignment and batch decoupling for horizontal scalability, but not load-tested"
"robust"	"What failure modes did you handle?"	You can answer this well — 10 failure scenarios
"enterprise-grade"	Everything	Do not say this
"state-of-the-art"	"Compared to what?"	Do not say this

SECTION 15: CODE AND ARTIFACT AREAS TO KNOW COLD

Priority ordering: *P0 = you will almost certainly be asked. P1 = likely in technical rounds. P2 = deep-dive or system design context.*

15.1 P0 — KNOW COLD (High Probability Questions)

P0-A: SHA-256 Deterministic Assignment

File: `scripts/generate_assignments_v0.py`

Know:

- The exact formula: `bucket = int(SHA256(f"{exp_key}:{user_id}"))[:12], 16) / (16**12)`
- Why 12 hex chars: gives ~281 trillion buckets, practically continuous [0,1]

- Why deterministic matters: same user always in same variant, no random re-assignment between runs
- Why not `random.random()`: non-reproducible, session-dependent

Practice question: "Walk me through how you assign users to variants and why you chose this approach."

P0-B: CUPED Formula and θ Calculation

File: `scripts/run_cuped_v0.py`

Know:

- $Y_{adj} = Y_{post} - \theta \times (X_{pre} - \text{mean}(X_{pre}))$
- $\theta = \text{Cov}(Y_{post}, X_{pre}) / \text{Var}(X_{pre})$
- Why this reduces variance: removes pre-existing between-user differences
- Actual θ in experiment: 0.00378
- Actual VR: 0.44% (low because RetailRocket dates are historical, not matched pre/post)
- What "variance reduction" means in practice: smaller confidence intervals, detect smaller effects

Practice question: "Explain CUPED to me like I'm a PM, then like I'm a stats PhD."

P0-C: SRM Detection Logic

File: `scripts/run_srm_check_v0.py`

Know:

- Chi-square goodness-of-fit, $df = k-1$ (for 2 variants, $df=1$)
- Expected ratio is 50/50
- Threshold: $p < 0.001$ (not 0.05 — more conservative to avoid false SRM flags)
- What causes SRM: client-side filtering, bots, deduplication errors, redirect issues
- What you do when SRM detected: pause experiment, do not analyze results

Practice question: "What is SRM, why does it matter, and how did you detect it?"

P0-D: Guardrail Decision Logic

File: `scripts/run_guardrail_decision_v0.py`

Know:

- Hierarchy: HOLD > SHIP > ITERATE > MONITOR
- HOLD triggered when: any guardrail metric exceeds its tolerance
- In the actual experiment: refund rate breached 3.1% vs 2.0% tolerance → HOLD
- Guardrail-first means: even if primary metric is positive, HOLD if guardrail breaches
- DecisionLog entry written to DB on every evaluation

Practice question: "Your experiment shows a 420% lift on the primary metric but a guardrail is breached. What do you do?"

P0-E: Batch-Authoritative Design

Concept — no single file

Know:

- Batch = the source of truth for final decisions
- Streaming = provisional signals only, never authoritative

- Why: streaming can have deduplication errors, ordering issues, incomplete windows
- Stream-batch delta in this system: 6% (batch = streaming × 0.94)
- This is a deliberate architectural choice, not a limitation

Practice question: "How does your system handle the relationship between streaming data and batch data?"

15.2 P1 — LIKELY IN TECHNICAL ROUNDS

P1-A: A/A Validation Design

File: `scripts/run_cuped_aa_validation_v0.py`

Know: 1000 runs, FPR=5.5%, KS stat=0.019 vs critical=0.043, what each number means

P1-B: Five CUPED Edge Cases

File: `scripts/run_prd_core_validation_suite_v1.py` → `apply_cuped_with_edge_cases()`

Know: strong (VR=50.7%), weak→fallback, missing→fallback, zero-variance→fallback, imbalanced arms

P1-C: Three Causal Traps

File: `scripts/run_prd_core_validation_suite_v1.py` → `run_causal_trap_detectors()`

Know: peeking (days 3/7/10 p-values), novelty (7× week1/week2 ratio), Simpson's (overall+3.1% but segments negative)

P1-D: Right-Censoring Logic

File: `scripts/run_post_launch_validation_v0.py`

Know: maturity_window=30 days, readout=14 days → `right_censoring_flag=true`. Why: users who joined late haven't had full exposure window.

P1-E: Dead Letter Queue

File: `src/metasignal/streaming/streaming_prd_v1.py`

Know: 3 error codes: `MISSING_REQUIRED_FIELD`, `INVALID_VARIANT`, `SCHEMA_TYPE_MISMATCH`. DLQ allows replay and forensics.

P1-F: Alembic Migration Structure

Files: Two migration files in `alembic/versions/`

Know: First migration: 8 tables. Second migration: `experiment_assignments` (foreign key to `experiments` + `users`). Why two: incremental schema evolution pattern.

P1-G: DOW Anomaly Detection

File: `scripts/run_anomaly_detector_v0.py`

Know: Compare day's metric to rolling average of same weekday. Z-score threshold. Without DOW adjustment, Mondays look like anomalies because Monday traffic differs from Friday.

15.3 P2 — DEEP DIVE / SYSTEM DESIGN

P2-A: 13 Operational Scenarios

File: `scripts/seed_operational_history_v1.py`

Know: SIM-001 through SIM-013, their days, their types. Most important: iOS outage (day 28), human override (day 44), clean ship (day 50), DOW false positive (day 35).

P2-B: 15 Streaming Classes

File: `src/metasignal/streaming/streaming_prd_v1.py`

Know: Class names and responsibilities. Core chain: `EventSource` → `AsyncEventQueue` → `StreamEventConsumer` → `[validators/monitors/detectors]`. `ReplayManager` handles DLQ replay.

P2-C: FastAPI Endpoint Map

File: `src/metasignal/api/app.py`

Know: 10 endpoints, their methods, what each does. Most important: `POST /decisions/validate-override` → requires `override_reason` or returns HTTP 422.

P2-D: PRD Completion Map

File: `outputs/reports/metasignal_prd_completion_report_v1.json`

Know: 13 core items, 21 streaming items, all passing. This is your proof of completeness.

P2-E: Golden Scenarios (v1)

File: `scripts/run_prd_core_validation_suite_v1.py` → `run_golden_scenarios_v1()`

Know: 12 scenarios including the actual experiment (lift=420%, p=5e-44, HOLD due to guardrail), right-censoring, novelty, Simpson's, peeking.

15.4 ARTIFACTS TO HAVE OPEN DURING INTERVIEW (IF ALLOWED)

Priority order if you can share screen or reference:

`outputs/reports/metasignal_prd_completion_report_v1.json` — proof of completeness
`outputs/validation/cuped_aa_validation_results.json` — A/A numbers
`outputs/evidence/anomaly_detection_results.json` — backtest precision/recall
`outputs/streaming/scenario_report.json` — streaming coverage
`outputs/validation/streaming_prd_v1_validation.json` — 43/43

15.5 STUDY SEQUENCE (3-DAY PLAN)

Day 1 — Statistical Core:

- Re-read `run_cuped_v0.py` line by line
- Re-read `run_srm_check_v0.py`
- Re-read `run_guardrail_decision_v0.py`
- Practice explaining CUPED to a non-statistician and a statistician
- Drill A/A validation numbers until instant recall

Day 2 — Architecture and Data:

- Re-read `seed_operational_history_v1.py` — all 13 scenarios

- Re-read `run_prd_core_validation_suite_v1.py` — all four functions
- Sketch the DB schema from memory (13 tables, which are Alembic vs raw SQL)
- Sketch the streaming class hierarchy from memory

Day 3 — Positioning and Edge Cases:

- Practice all five CUPED edge cases out loud
- Practice three causal trap explanations
- Do mock answer to: "What would you do differently if this were production?"
- Review Section 14 (Do Not Say) — internalize all substitutions

SECTION 16: ONE-PAGE INTERVIEW CHEAT SHEET

Print this or memorize it. These are your fastest-recall reference points.

IDENTITY STATEMENT

What is MetaSignal?

"A batch-authoritative, production-simulated A/B testing platform. Solo-built. Public/synthetic data only. Not deployed. Covers the full experiment lifecycle."

One-sentence positioning:

"I built everything that sits between 'we have a hypothesis' and 'we have a decision' in an experimentation system."

CRITICAL NUMBERS

Fact	Value
Dataset	RetailRocket, 2,756,101 rows
Users assigned	20,000 (SHA-256)
DB tables	13 (9 Alembic + 4 raw SQL)
Alembic migrations	2
FastAPI endpoints	10 (8 smoke-tested)
PRD items	34 (13 core + 21 streaming) — all pass
Streaming classes	15
Streaming scenarios	10
Streaming checks	43/43 pass
A/A runs	1,000
A/A FPR	5.5%
KS stat / critical	0.019 / 0.043 (pass)

Fact	Value
CUPED edge cases	5
Strong-correlation VR	50.7%
Actual experiment VR	0.44%
Actual experiment θ	0.00378
Actual lift	420% (synthetic uplift, 255 users)
Actual p-value	5e-44
Decision	HOLD (guardrail breach)
Guardrail threshold	2.0% tolerance
Observed refund rate	3.1%
Operational scenarios	13 (SIM-001..SIM-013)
Operational history	60 days
Anomaly backtest precision	1.0
Anomaly backtest recall	0.87
DOW false positives suppressed	3
DLQ error codes	3
Stream-batch delta	6%
CI platform	GitHub Actions, ubuntu-latest, Python 3.11
Override enforcement	HTTP 422 without override_reason

KEY FORMULAS

SHA-256 Assignment:

```
bucket = int(SHA256(f"{exp_key}:{user_id}")[12], 16) / (16**12)
```

CUPED:

$$Y_{adj} = Y_{post} - \theta \times (X_{pre} - \text{mean}(X_{pre}))$$

$$\theta = \text{Cov}(Y_{post}, X_{pre}) / \text{Var}(X_{pre})$$

SRM: Chi-square GOF, df=1, p<0.001 threshold

DOW Anomaly: Z-score of today vs rolling same-weekday average

TRUTH LABELS (INSTANT RECALL)

Claim	Label
PostgreSQL + SQLAlchemy + Alembic	IMPLEMENTED

Claim	Label
SHA-256 deterministic assignment	IMPLEMENTED
CUPED with 5 edge cases	IMPLEMENTED
SRM chi-square detection	IMPLEMENTED
Guardrail-first decision logic	IMPLEMENTED
A/A validation (1000 runs)	IMPLEMENTED
10 FastAPI endpoints	IMPLEMENTED
asyncio streaming (15 classes)	IMPLEMENTED
13 operational scenarios	IMPLEMENTED
Causal trap detectors	IMPLEMENTED
Right-censoring detection	IMPLEMENTED
Kafka / real streaming	NOT PRESENT
Production deployment	NOT PRESENT
Real user data	NOT PRESENT
Live monitoring	NOT PRESENT

FIVE DANGER ZONES + SAFE EXITS

"Is this production?" → "No — production-simulated. Designed with production patterns; never deployed."

"How does it scale?" → "Stateless assignment, batch decoupled from streaming — horizontally scalable by design, not load-tested."

"Why 5.5% FPR in A/A?" → "Expected — that's close to the 5% theoretical false positive rate under the null hypothesis. The distribution passes the KS test."

"What's the 420% lift?" → "Synthetic — I injected 255 users with deterministic uplift to validate the pipeline can detect true effects. The number is not a real business outcome."

"Why asyncio and not Kafka?" → "Scope decision. I wanted to build the streaming logic patterns — event processing, DLQ, reconciliation — without the operational overhead of a Kafka cluster. The architecture is designed to swap in a real broker."

THE ARCHITECTURE IN 4 LINES

```
RetailRocket (2.7M rows) → PostgreSQL (13 tables) → Batch analysis (CUPED/SRM/guardrails) ■ asyncio stream
(15 classes) → provisional signals → reconcile → batch Operational history seeder (60 days / 13
scenarios) → anomaly detection → decision audit FastAPI (10 endpoints) → expose decisions + overrides +
health
```

SECTION 17: FINAL VERDICT

17.1 THE HONEST ASSESSMENT

MetaSignal is a **high-signal, high-credibility portfolio project** with specific, well-defined constraints. The correct frame is not "I built a production system" — it is "I built a technically rigorous system that demonstrates I understand what production systems do and why."

What MetaSignal proves, clearly and defensibly:

- You understand the full experimentation lifecycle end-to-end, not just statistics in isolation
- You can translate statistical concepts (CUPED, SRM, chi-square) into working code
- You designed a database schema appropriate for an experimentation platform
- You thought about edge cases (5 CUPED edge cases, 10 streaming scenarios, 13 operational scenarios)
- You understand the distinction between streaming data and batch data and why it matters for decisions
- You can implement and validate anomaly detection with real backtest metrics
- You wrote a spec (PRD), built against it, and validated compliance — a professional engineering practice
- You built governance mechanisms (override enforcement, audit trails) that go beyond toy projects

What MetaSignal does not prove:

- You can operate a system under production load
- You can debug production incidents
- You have experience with distributed systems at scale
- You have collaborated on a large codebase with multiple engineers
- Your statistical choices would survive a production peer review

17.2 HOW STRONG IS THIS FOR EACH INTERVIEW TYPE?

Interview Type	MetaSignal Strength	Notes
Data Science / Experimentation	Very Strong	Full lifecycle, real statistical rigor
Analytics Engineering	Strong	DB schema, ORM, Alembic migrations, data quality
ML Engineering	Moderate	No ML model — only statistical methods
Software Engineering (general)	Moderate	Good architecture, but limited scale evidence
Data Engineering	Moderate	Pipeline patterns present; no Spark/Airflow/cloud
Product Analytics	Very Strong	Causal traps, guardrail logic, decision framework
Experimentation Platform (Meta/Google)	Strong with caveats	Shows depth; must be honest about production gap
Startup (first data hire)	Very Strong	Solo-built, end-to-end, practical

17.3 THE STRONGEST HONEST NARRATIVE

"I built MetaSignal to go deep on experimentation infrastructure — the kind of system I'd be working on and with in this role. I didn't build a tutorial project; I wrote a PRD, implemented against it, created failure scenarios, and validated every component. It's not production, and I'm not pretending it is. But I can tell you exactly how every part works, defend every design decision, and explain what I'd do differently at scale. The goal was to demonstrate I can think at the level of the system, not just at the level of a notebook."

17.4 INTERVIEWERS' LIKELY REACTIONS

Reaction 1 (Positive): "This person clearly knows what they're building. The detail level is impressive — most candidates couldn't describe CUPED implementation specifics or explain why SRM threshold is $p < 0.001$ rather than $p < 0.05$."

Reaction 2 (Skeptical): "This is all simulation. How do I know you can handle production messiness — schema migrations under live traffic, oncall, latency SLOs?"

Your response to Reaction 2: "That's a fair point. The simulation was intentional — I wanted to master the logic without the noise. Production experience is what I'm looking to build in this role. What I can promise is that I've already done the work of understanding *why* each piece exists, which means I'll ramp faster."

Reaction 3 (Curious): "Tell me more about the streaming piece."

Your response: "The streaming layer is asyncio-based — I want to be upfront, not Kafka. 15 classes, 10 failure injection scenarios, 43/43 validation checks. The design is intentionally batch-authoritative — streaming is provisional, never decision-authoritative. Stream-batch reconciliation runs at the end of each window and I found a 6% delta in my simulation, which is realistic for real systems."

17.5 WHAT TO BUILD NEXT (IF TIME ALLOWS)

In priority order, these additions would significantly strengthen the portfolio signal:

Add a proper README with architecture diagram — most interviewers open GitHub before the call

Add a Jupyter notebook showing the CUPED derivation and A/A results with charts — visual signal for analytics roles

Replace asyncio with a real Kafka setup (even Docker Compose locally) — eliminates the biggest credibility gap

Write one proper blog post about the SRM detection or CUPED implementation — demonstrates communication skill

Add a simple Docker Compose for the full stack (PostgreSQL + API) — makes it runnable for reviewers

Document one real-world incident analogy per scenario — connects simulation to production thinking

17.6 ONE-PARAGRAPH FINAL VERDICT

MetaSignal is an unusually rigorous portfolio project that demonstrates experimentation system thinking at a level rare among self-taught or early-career candidates. The statistical implementations are correct, the architectural decisions are principled, and the validation approach is professional. The core constraint — simulation only, no production deployment — is genuine but clearly scoped and consistently acknowledged. Candidates who present this project well, without overclaiming, with precise numbers recalled from memory, and with the ability to explain every design decision under cross-examination, will make a strong impression in experimentation, analytics engineering, and product data science interviews. The project rewards preparation: its strength scales directly with how deeply you know it.

End of MetaSignal Ground Truth / Interview Defense Master Pack v2.0 — Part 3

Sections 13–17 complete. Combine with Part 1 (Sections 0–6) and Part 2 (Sections 7–12) for full document.